Audio & Speech Processing

Spectrogram, Whisper, ASR, TTS, Transformer, Pipeline

Hanwha Life Vietnam

Phiên bản v1.0.0

Ngày 13 tháng 10 năm 2026

Mục lục

1 Tổng quan **2**
- 1.1 Triết lý trình bày . . . 2
- 1.2 Vì sao âm thanh đáng học riêng . . . 2
- 1.3 Cách đọc tài liệu . . . 2
- 1.4 Nội dung các chương . . . 3
- 1.5 Nguồn và quy ước . . . 3

2 Làm việc với dữ liệu âm thanh **4**
- 2.1 Từ sóng âm đến dãy số . . . 4
  - 2.1.1 Tần số lấy mẫu . . . 4
  - 2.1.2 Biên độ và độ sâu bit . . . 5
- 2.2 Ba cách nhìn một tín hiệu âm thanh . . . 5
  - 2.2.1 Dạng sóng – miền thời gian . . . 6
  - 2.2.2 Phổ tần số – miền tần số . . . 7
  - 2.2.3 Spectrogram – tần số thay đổi theo thời gian . . . 7
  - 2.2.4 Phổ mel – theo cảm nhận của tai người . . . 8
- 2.3 Nạp và khám phá bộ dữ liệu . . . 8
- 2.4 Tiền xử lý: chuẩn bị dữ liệu cho mô hình . . . 9
  - 2.4.1 Đổi tần số lấy mẫu . . . 9
  - 2.4.2 Lọc theo tiêu chí . . . 10
  - 2.4.3 Trích đặc trưng cho mô hình . . . 10

3 Ứng dụng âm thanh với `pipeline` **12**
- 3.1 Ý tưởng của `pipeline` . . . 12
- 3.2 Phân loại âm thanh . . . 12
- 3.3 Nhận dạng tiếng nói . . . 13
- 3.4 Sinh âm thanh: giọng nói và nhạc . . . 14
  - 3.4.1 Tổng hợp giọng nói . . . 14
  - 3.4.2 Sinh nhạc . . . 14

4 Kiến trúc Transformer cho âm thanh **16**
- 4.1 Nhắc lại Transformer trong 60 giây . . . 16
- 4.2 Đưa âm thanh vào và lấy kết quả ra . . . 16
  - 4.2.1 Đầu vào mô hình . . . 17
  - 4.2.2 Đầu ra mô hình . . . 17
- 4.3 Kiến trúc 1 – CTC: ASR bằng encoder . . . 17
  - 4.3.1 Vấn đề căn chỉnh và cách CTC giải quyết . . . 17
- 4.4 Kiến trúc 2 – Seq2seq: encoder-decoder . . . 19
  - 4.4.1 ASR với seq2seq (Whisper) . . . 19
  - 4.4.2 TTS với seq2seq (SpeechT5) . . . 19
- 4.5 Kiến trúc 3 – Phân loại âm thanh . . . 20

4.5.1 Coi spectrogram như một bức ảnh . . . 20
4.5.2 Mọi encoder đều có thể thành bộ phân loại . . . 20

5 Dựng bộ phân loại thể loại nhạc 22
5.1 Các bài toán phân loại âm thanh và mô hình có sẵn . . . 22
5.1.1 Phát hiện từ khoá (keyword spotting) . . . 22
5.1.2 Nhận diện ngôn ngữ (LID) . . . 23
5.1.3 Phân loại không mẫu (zero-shot) với CLAP . . . 23
5.2 Tinh chỉnh mô hình phân loại nhạc . . . 23
5.2.1 Bộ dữ liệu GTZAN . . . 24
5.2.2 Tiền xử lý: trích đặc trưng và chuẩn hoá . . . 24
5.2.3 Huấn luyện với `Trainer` . . . 25
5.3 Dựng demo với Gradio . . . 26

6 Nhận dạng tiếng nói tự động 28
6.1 Hai họ mô hình ASR . . . 28
6.1.1 Mổ xẻ mô hình CTC (Wav2Vec2) . . . 28
6.1.2 Bước tiến seq2seq: Whisper . . . 28
6.1.3 Dịch giọng nói và đa ngôn ngữ . . . 29
6.1.4 Chép âm thanh dài và dấu thời gian . . . 29
6.2 Chọn bộ dữ liệu . . . 30
6.3 Đo chất lượng: tỉ lệ lỗi từ (WER) . . . 30
6.4 Tinh chỉnh Whisper cho một ngôn ngữ mới . . . 32
6.4.1 Nạp dữ liệu và processor . . . 32
6.4.2 Tiền xử lý . . . 33
6.4.3 Collator, thước đo và cấu hình huấn luyện . . . 33
6.5 Dựng demo . . . 35

7 Từ văn bản sang tiếng nói 36
7.1 Vì sao TTS khó . . . 36
7.2 Bộ dữ liệu cho TTS . . . 36
7.3 SpeechT5: encoder-decoder + vocoder . . . 37
7.4 Bark: sinh thẳng sóng âm . . . 38
7.5 MMS: hơn 1 100 ngôn ngữ . . . 38
7.6 Đánh giá mô hình TTS . . . 39

8 Ghép mọi thứ lại 41
8.1 Dịch giọng-sang-giọng . . . 41
8.1.1 Tầng 1 – dịch giọng . . . 41
8.1.2 Tầng 2 – tổng hợp giọng . . . 42
8.1.3 Ghép lại và dựng demo . . . 42
8.2 Trợ lý giọng nói Marvin . . . 43
8.2.1 Tầng 1 – phát hiện từ đánh thức . . . 43
8.2.2 Tầng 2 – chép lời câu hỏi . . . 43
8.2.3 Tầng 3 – hỏi mô hình ngôn ngữ . . . 44
8.2.4 Tầng 4 – tổng hợp giọng trả lời . . . 44
8.3 Chép lời cuộc họp . . . 44
8.3.1 Phân tách người nói . . . 45
8.3.2 Chép lời kèm dấu thời gian . . . 46
8.3.3 Khớp hai mốc thời gian với Speechbox . . . 46

9 Case study: kiểm tra đọc đúng script **47**
9.1 Phát biểu bài toán . . . 47
9.2 Khâu 1 – Nhận dạng tiếng nói tiếng Việt . . . 48
9.2.1 Chọn cỡ mô hình và chuẩn bị âm thanh . . . 48
9.2.2 Khi nào cần fine-tune, khi nào không . . . 48
9.3 Khâu 2 – So khớp script bằng LLM . . . 50
9.3.1 Hai kiểu kiểm tra . . . 50
9.3.2 Gọi LLM và yêu cầu trả JSON có cấu trúc . . . 50
9.3.3 Ghép hai khâu . . . 51
9.4 Kiến trúc triển khai và lưu ý vận hành . . . 51

Chương 1

Tổng quan

Tài liệu này là bộ ghi chú học tập về **xử lý âm thanh và tiếng nói** bằng học sâu, biên soạn dựa trên khoá *Audio Course* của Hugging Face. Tài liệu đi theo hướng **high-level, trực quan và thực chiến**: ưu tiên hiểu ý tưởng và biết cách dùng thư viện (`transformers`, `datasets`, `librosa`, `gradio`) thay vì tự cài đặt thuật toán xử lý tín hiệu hay huấn luyện mô hình từ con số không.

1.1 Triết lý trình bày

- **Trực giác trước, chi tiết sau** – mỗi khái niệm được giải thích bằng ví dụ đời thường và sơ đồ. Thay cho công thức toán, tài liệu mô tả bằng lời: nói rõ đại lượng nào đi vào, đại lượng nào đi ra, và vì sao.
- **Thực chiến bằng thư viện** – code ví dụ dùng hệ sinh thái Hugging Face cùng các công cụ âm thanh phổ biến. Mục tiêu là chạy được việc thật: nạp dữ liệu, nhận dạng tiếng nói, tổng hợp giọng nói, phân loại âm thanh.
- **Sơ đồ hoá** – kiến trúc mô hình, luồng xử lý và quan hệ giữa các thành phần đều được minh hoạ bằng sơ đồ để dễ nhớ và dễ tra cứu.

1.2 Vì sao âm thanh đáng học riêng

Âm thanh khác văn bản ở một điểm cốt lõi: nó là *tín hiệu liên tục theo thời gian*. Một đoạn ghi âm không phải là chuỗi "từ" rời rạc mà là hàng chục nghìn con số đo biên độ sóng âm mỗi giây. Trước khi mô hình học được, ta phải biến dòng tín hiệu đó thành dạng số có cấu trúc, rồi thường chuyển sang **phổ** (spectrogram) – một "bức ảnh" cho biết các tần số mạnh yếu ra sao theo thời gian. Nắm chắc bước chuyển đổi này là chìa khoá để hiểu mọi mô hình âm thanh về sau.

1.3 Cách đọc tài liệu

Mỗi chương gồm ba lớp nội dung đan xen:

- **Lý thuyết** – giải thích bằng ví dụ và sơ đồ, hạn chế ký hiệu toán.
- **Code** – ví dụ Python đặt trong khung, có chú thích tiếng Việt, kèm giải thích từng bước dùng để làm gì.
- **Áp dụng** – checklist, bảng so sánh, lưu ý khi đưa vào dự án thực tế.

1.4 Nội dung các chương

Chương	Nội dung chính
Ch. 2: Làm việc với dữ liệu âm thanh	tín hiệu số, tần số lấy mẫu, biên độ, dạng sóng, phổ và phổ mel; nạp, khám phá và tiền xử lý một bộ dữ liệu âm thanh.
Ch. 3: Ứng dụng âm thanh với `pipeline`	phân loại âm thanh, nhận dạng tiếng nói, tổng hợp giọng nói chỉ với vài dòng code.
Ch. 4: Kiến trúc Transformer cho âm thanh	cách đưa âm thanh vào Transformer, ba kiểu mô hình: CTC, chuỗi-sang-chuỗi (seq2seq) và phân loại.
Ch. 5: Dựng bộ phân loại thể loại nhạc	chọn mô hình, tinh chỉnh trên bộ GTZAN, dựng demo bằng Gradio.
Ch. 6: Nhận dạng tiếng nói tự động	họ mô hình ASR, chọn dữ liệu, cách đo lỗi (WER/CER), tinh chỉnh Whisper, dựng demo.
Ch. 7: Từ văn bản sang tiếng nói	bộ dữ liệu TTS, mô hình tiền huấn luyện (SpeechT5, Bark, MMS), tinh chỉnh và cách đánh giá.
Ch. 8: Ghép mọi thứ lại	dịch giọng-sang-giọng, trợ lý giọng nói, chép lời cuộc họp.
Ch. 9: Case study kiểm tra script	kiểm tra khách đọc đúng script bảo hiểm: Whisper tiếng Việt + so khớp bằng LLM, khi nào cần fine-tune, lưu ý quyền riêng tư.

1.5 Nguồn và quy ước

Nội dung được biên soạn và diễn giải lại từ khoá học *Audio Course* của Hugging Face (`https://huggingface.co/learn/audio-course`). Các đoạn mô tả đã được viết lại bằng tiếng Việt cho phù hợp mục đích học tập; phần code giữ nguyên cú pháp cần thiết để chạy được. Mọi con số minh hoạ (ví dụ điểm số, nhãn dự đoán, kết quả bản ghi) đều lấy trực tiếp từ ví dụ trong tài liệu gốc. Riêng Chương 9 là một case study ứng dụng mở rộng *không* thuộc khoá gốc; chương đó không đưa ra số liệu đo đạc mà để các ngưỡng dưới dạng tham số kèm hướng dẫn tự hiệu chỉnh.

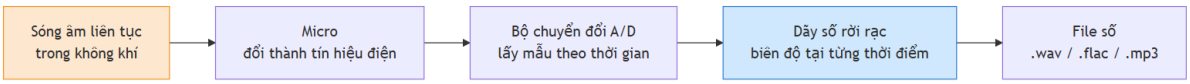
Chương 2

Làm việc với dữ liệu âm thanh

Chương này xây nền tảng: âm thanh được số hoá ra sao, các cách nhìn một tín hiệu âm thanh, và quy trình nạp – khám phá – tiền xử lý một bộ dữ liệu bằng thư viện `datasets`. Hiểu chắc phần này giúp mọi chương sau trở nên dễ chịu, vì mọi mô hình âm thanh đều bắt đầu từ cùng một dạng dữ liệu đầu vào.

2.1 Từ sóng âm đến dãy số

Sóng âm vốn là **tín hiệu liên tục**: trong một khoảng thời gian bất kỳ, nó có vô số giá trị. Máy tính thì chỉ làm việc với *dãy số hữu hạn*. Vì vậy sóng âm phải được chuyển thành **biểu diễn số** trước khi lưu trữ hay xử lý. Luồng chuyển đổi được minh hoạ ở Hình 2.1: micro đổi sóng âm thành tín hiệu điện, rồi bộ chuyển đổi tương tự–số (A/D) "chụp" giá trị tín hiệu tại những mốc thời gian đều đặn.



Hình 2.1: Từ sóng âm liên tục đến file số qua quá trình lấy mẫu.

Các bộ dữ liệu âm thanh thường gặp định dạng `.wav`, `.flac` hay `.mp3`. Chúng chủ yếu khác nhau ở cách nén biểu diễn số của tín hiệu.

2.1.1 Tần số lấy mẫu

Lấy mẫu là đo giá trị tín hiệu tại những mốc thời gian cách đều nhau. **Tần số lấy mẫu** là số mẫu lấy được trong một giây, đo bằng hertz (Hz). Vài mốc để tham chiếu:

- Chất lượng CD: 44 100 Hz (lấy mẫu 44 100 lần mỗi giây).
- Âm thanh độ phân giải cao: 192 000 Hz.
- Huấn luyện mô hình tiếng nói: thường dùng 16 000 Hz (16 kHz).

Tần số lấy mẫu quyết định *tần số cao nhất* có thể ghi lại được, và trần đó bằng đúng một nửa tần số lấy mẫu (giới hạn Nyquist). Vì âm tiếng nói của người chủ yếu nằm dưới 8 kHz, lấy mẫu ở 16 kHz là đủ. Dùng tần số cao hơn không thêm thông tin mà chỉ tốn tính toán; ngược lại, lấy mẫu quá thấp sẽ **mất thông tin** – tiếng nói ở 8 kHz nghe bị bí vì mất phần tần số cao.

Nguyên tắc vàng về tần số lấy mẫu

Mọi mẫu âm thanh trong bộ dữ liệu phải cùng một tần số lấy mẫu. Khi tinh chỉnh mô hình tiền huấn luyện, tần số lấy mẫu của dữ liệu của bạn *phải khớp* với tần số mô hình đã học. Một đoạn 5 giây ở 16 kHz là 80 000 giá trị; cũng 5 giây đó ở 8 kHz chỉ còn 40 000 giá trị. Transformer coi mỗi mẫu là một chuỗi, nên chuỗi dài khác nhau khiến mô hình khó khái quát giữa các tần số lấy mẫu khác nhau.

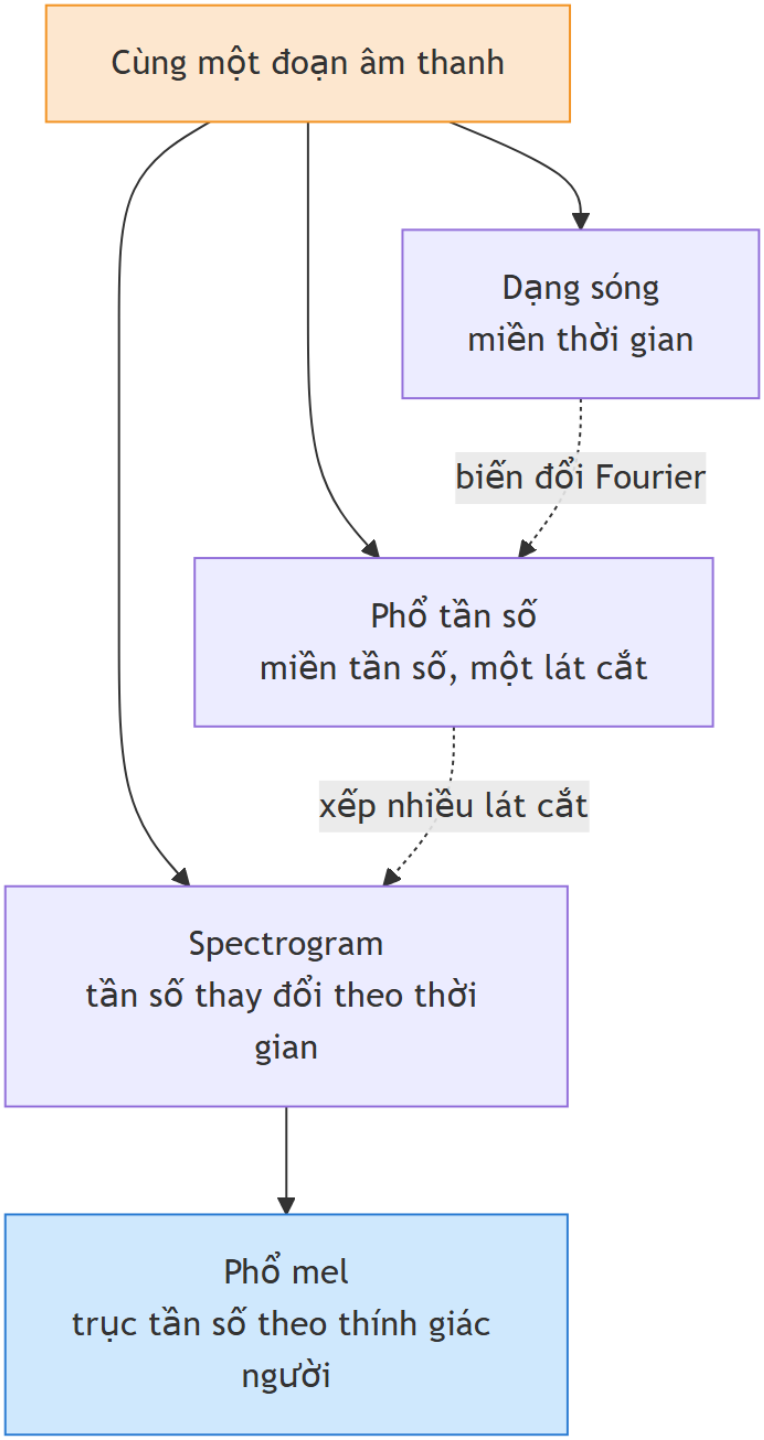
2.1.2 Biên độ và độ sâu bit

Nếu tần số lấy mẫu cho biết *đo bao lâu một lần*, thì **biên độ** là *đo cái gì*: nó mô tả mức áp suất âm tại từng thời điểm, mà tai ta cảm nhận là *độ to*. Biên độ thường tính bằng decibel (dB). Giọng nói bình thường dưới 60 dB, còn một buổi hoà nhạc rock có thể quanh 125 dB.

Độ sâu bit quyết định độ chính xác khi ghi lại giá trị biên độ. Càng nhiều bit, biểu diễn số càng bám sát sóng âm gốc. Hai mức phổ biến là 16-bit và 24-bit – tức có 65 536 hoặc 16 777 216 nấc để lượng tử hoá biên độ. Việc làm tròn về nấc gần nhất sinh ra "nhiễu lượng tử", nhưng với 16-bit nhiễu này đã nhỏ tới mức không nghe thấy. Riêng âm thanh 32-bit lưu mẫu dưới dạng số thực (dấu phẩy động) trong khoảng từ $-1{,}0$ đến $1{,}0$. Vì mô hình học máy làm việc tự nhiên với số thực, âm thanh luôn được đổi về dạng số thực trước khi huấn luyện.

2.2 Ba cách nhìn một tín hiệu âm thanh

Cùng một đoạn âm thanh có thể được nhìn theo nhiều cách, mỗi cách làm nổi bật một khía cạnh khác nhau (Hình 2.2). Quan trọng: dạng sóng và phổ tần số chứa *cùng một lượng thông tin*, chỉ là hai góc nhìn khác nhau của cùng dữ liệu.



Hình 2.2: Các cách biểu diễn cùng một tín hiệu âm thanh.

2.2.1 Dạng sóng – miền thời gian

Dạng sóng vẽ giá trị các mẫu theo thời gian, cho thấy biên độ thay đổi ra sao. Nó hữu ích để nhận ra thời điểm các sự kiện âm thanh, độ to tổng thể, và các bất thường hay nhiễu. Thư viện `librosa` vẽ dạng sóng rất gọn:

```
import librosa
import matplotlib.pyplot as plt
```

```
import librosa.display

# Nạp một âm thanh mẫu đi kèm thư viện: tiếng kèn trumpet
array, sampling_rate = librosa.load(librosa.ex("trumpet"))

plt.figure().set_figwidth(12)
librosa.display.waveshow(array, sr=sampling_rate)
# Trục đứng: biên độ (trong khoảng -1.0 đến 1.0); trục ngang: thời gian
```

Mỗi điểm trên đồ thị ứng với một giá trị mẫu. Nghe kèm nhìn dạng sóng là cách tốt để hiểu dữ liệu, kiểm tra xem các bước tiền xử lý (chuẩn hoá, đổi tần số lấy mẫu, lọc) đã được áp dụng đúng chưa.

2.2.2 Phổ tần số – miền tần số

Phổ tần số cho biết tín hiệu được tạo nên từ những tần số nào và mỗi tần số mạnh ra sao. Nó được tính bằng *biến đổi Fourier rời rạc* (DFT) – trên máy tính luôn chạy bằng thuật toán nhanh FFT. Thay vì phân tích cả đoạn, ta thường lấy một vùng nhỏ để quan sát:

```
import numpy as np

dft_input = array[:4096]           # ~ độ dài nốt nhạc đầu tiên

# Nhân với cửa sổ Hann rồi tính DFT bằng FFT
window = np.hanning(len(dft_input))
dft = np.fft.rfft(dft_input * window)

# Lấy độ lớn rồi đổi sang thang decibel cho dễ nhìn
amplitude = np.abs(dft)
amplitude_db = librosa.amplitude_to_db(amplitude, ref=np.max)
frequency = librosa.fft_frequencies(sr=sampling_rate, n_fft=len(dft_input))

plt.figure().set_figwidth(12)
plt.plot(frequency, amplitude_db)
plt.xscale("log")                  # trục tần số thường vẽ theo thang log
```

Các đỉnh trong phổ ứng với các hoạ âm (harmonic) của nốt đang chơi; hoạ âm cao thường nhẹ hơn. Đầu ra DFT là số phức, nhưng với học máy ta thường chỉ lấy *độ lớn* (biên độ) và bỏ phần pha.

2.2.3 Spectrogram – tần số thay đổi theo thời gian

Phổ tần số chỉ là một "ảnh chụp đứng yên" tại một thời điểm. Nếu muốn thấy tần số *thay đổi theo thời gian*, ta cắt âm thanh thành nhiều lát mỏng, tính phổ cho từng lát rồi xếp chúng lại cạnh nhau. Kết quả là **spectrogram**, cho phép nhìn đồng thời cả thời gian, tần số và biên độ trên một hình. Thuật toán thực hiện gọi là biến đổi Fourier thời gian ngắn (STFT).

```
D = librosa.stft(array)
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)

plt.figure().set_figwidth(12)
librosa.display.specshow(S_db, x_axis="time", y_axis="hz")
plt.colorbar()
# Trục ngang: thời gian; trục đứng: tần số (Hz); màu: độ mạnh (dB)
```

Spectrogram là một trong những công cụ giàu thông tin nhất: với nhạc, ta thấy các nhạc cụ và giọng hát; với tiếng nói, ta phân biệt được các nguyên âm. Vì spectrogram và dạng sóng là hai góc nhìn của cùng

dữ liệu, có thể dựng lại dạng sóng từ spectrogram bằng STFT ngược – nhưng cần cả thông tin pha. Khi spectrogram do mô hình sinh ra (thường chỉ có biên độ), ta cần thuật toán khôi phục pha như Griffin–Lim hoặc một mạng nơ-ron gọi là **vocoder**.

2.2.4 Phổ mel – theo cảm nhận của tai người

Tai người nhạy với thay đổi ở tần số thấp hơn là tần số cao, và độ nhạy giảm dần theo kiểu logarit. **Thang mel** là một thang cảm nhận mô phỏng đặc điểm này. **Phổ mel** giống spectrogram thường, nhưng trục tần số được uốn về thang mel qua một bộ lọc gọi là *mel filterbank*. Đây là dạng đầu vào phổ biến nhất cho mô hình tiếng nói.

```
S = librosa.feature.melspectrogram(
    y=array, sr=sampling_rate, n_mels=128, fmax=8000
)
S_dB = librosa.power_to_db(S, ref=np.max)   # đổi sang decibel -> log-mel

plt.figure().set_figwidth(12)
librosa.display.specshow(S_dB, x_axis="time", y_axis="mel",
                         sr=sampling_rate, fmax=8000)
plt.colorbar()
```

Ở đây `n_mels` là số dải mel (thường dùng 40 hoặc 80), `fmax` là tần số cao nhất ta quan tâm. Khi đổi độ mạnh sang decibel, ta có **log-mel spectrogram**. So với spectrogram thường, phổ mel nắm được nhiều đặc trưng có ý nghĩa với thính giác người hơn, nên rất được ưa dùng trong nhận dạng tiếng nói, nhận diện người nói và phân loại thể loại nhạc.

Lưu ý khi dùng phổ mel

Phổ mel không có một định nghĩa duy nhất: có hai thang mel thông dụng ("htk" và "slaney"), và việc đổi sang log-mel đôi khi chỉ lấy logarit chứ không ra decibel thật. Vì vậy nếu một mô hình yêu cầu phổ mel làm đầu vào, hãy chắc chắn bạn tính nó *đúng cách* mô hình mong đợi. Tạo phổ mel còn là phép biến đổi có mất mát: dựng lại dạng sóng từ phổ mel khó hơn, cần vocoder như HiFi-GAN.

2.3 Nạp và khám phá bộ dữ liệu

Ta dùng thư viện `datasets` để tải và chuẩn bị dữ liệu âm thanh. Chỉ một dòng `load_dataset` là đủ. Ví dụ dưới dùng bộ **MINDS-14** – ghi âm người hỏi hệ thống ngân hàng điện tử bằng nhiều ngôn ngữ và phương ngữ.

```
pip install datasets[audio] soundfile librosa torchcodec
```

```
from datasets import load_dataset

# Chỉ lấy phân nhánh tiếng Anh - Úc (en-AU), tập train
minds = load_dataset("PolyAI/minds14", name="en-AU", split="train")
minds
# Dataset có các cột: path, audio, transcription,
#   english_transcription, intent_class, lang_id -- 654 dòng
```

Mỗi dòng gồm file âm thanh, bản chép lời, bản dịch tiếng Anh và nhãn ý định (`intent_class`). Xem kỹ một ví dụ:

```
example = minds[0]
example
# audio -> {"path": ..., "array": array([...], float32),
#           "sampling_rate": 8000}
# transcription -> "I would like to pay my electricity bill ..."
# intent_class -> 13
```

Cột `audio` chứa ba trường quan trọng: `path` (đường dẫn file), `array` (dữ liệu âm thanh đã giải mã, một mảng NumPy một chiều) và `sampling_rate` (ở đây là 8 000 Hz). Nhãn ý định là số; đổi sang chuỗi có nghĩa bằng `int2str`:

```
id2label = minds.features["intent_class"].int2str
id2label(example["intent_class"])   # -> "pay_bill"
```

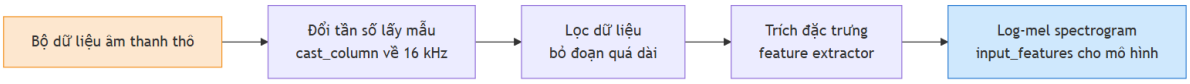
Nếu chỉ huấn luyện bộ phân loại, có thể bỏ bớt cột không cần (ví dụ `lang_id` giống nhau mọi dòng, `english_transcription` trùng với `transcription`):

```
minds = minds.remove_columns(["lang_id", "english_transcription"])
```

Muốn nghe thử vài mẫu, có thể dựng một demo nhỏ bằng Gradio để phát âm thanh ngẫu nhiên kèm nhãn ý định của chúng; hoặc vẽ dạng sóng của một ví dụ như ở Mục 2.2.1.

2.4 Tiền xử lý: chuẩn bị dữ liệu cho mô hình

Nạp xong mới là một nửa việc. Trước khi huấn luyện hay suy luận, dữ liệu cần qua ba bước (Hình 2.3): đổi tần số lấy mẫu, lọc, và trích đặc trưng.



Hình 2.3: Luồng tiền xử lý một bộ dữ liệu âm thanh.

2.4.1 Đổi tần số lấy mẫu

Phần lớn mô hình tiền huấn luyện mong đợi 16 kHz, nhưng MINDS-14 ở 8 kHz, nên cần *tăng mẫu* (upsample). Dùng `cast_column`: thao tác này không đổi file tại chỗ mà báo cho `datasets` đổi tần số lấy mẫu ngay khi nạp.

```
from datasets import Audio

minds = minds.cast_column("audio", Audio(sampling_rate=16_000))
minds[0]["audio"]["sampling_rate"]   # -> 16000
```

Vì sao đổi tần số lấy mẫu lại tế nhị

Tăng mẫu là ước lượng các giá trị nằm giữa những mẫu đã có (nội suy đường cong mượt giữa các điểm). **Giảm mẫu** khó hơn: không được "bỏ bớt mẫu" một cách thô bạo, vì sẽ sinh méo tín hiệu gọi là *aliasing*; phải lọc bỏ các tần số vượt giới hạn Nyquist mới trước. Việc này nên để các thư viện đã kiểm chứng kỹ như `librosa` hay `datasets` lo.

2.4.2 Lọc theo tiêu chí

Thường ta cần loại các đoạn quá dài để tránh tràn bộ nhớ khi huấn luyện. Vì bộ dữ liệu chưa có cột thời lượng, ta tạo cột tạm, lọc theo nó rồi xoá đi:

```
MAX_DURATION_IN_SECONDS = 20.0

def is_audio_length_in_range(input_length):
    return input_length < MAX_DURATION_IN_SECONDS

# Tạo cột thời lượng bằng librosa, lọc, rồi xoá cột tạm
new_column = [librosa.get_duration(y=x["array"], sr=x["sampling_rate"])
              for x in minds["audio"]]
minds = minds.add_column("duration", new_column)
minds = minds.filter(is_audio_length_in_range, input_columns=["duration"])
minds = minds.remove_columns(["duration"])
# Bộ dữ liệu giảm từ 654 xuống 624 ví dụ
```

2.4.3 Trích đặc trưng cho mô hình

Mô hình không nhận trực tiếp mảng mẫu thô mà cần **đặc trưng đầu vào** đúng định dạng. Mỗi mô hình có một lớp *feature extractor* riêng để chuyển đổi. Lấy Whisper (mô hình ASR của OpenAI) làm ví dụ, bộ trích đặc trưng của nó làm hai việc chính:

- **Chuẩn độ dài về 30 giây**: đoạn ngắn hơn được đệm thêm số không (im lặng) ở cuối; đoạn dài hơn bị cắt. Vì mọi mẫu đều dài bằng nhau, Whisper không cần mặt nạ chú ý (attention mask) – điểm đặc biệt so với phần lớn mô hình âm thanh khác.
- **Đổi sang log-mel spectrogram**: biến mảng âm thanh đã đệm thành phổ mel đo bằng decibel.

```
from transformers import WhisperFeatureExtractor

feature_extractor = WhisperFeatureExtractor.from_pretrained("openai/whisper-small")

def prepare_dataset(example):
    audio = example["audio"]
    features = feature_extractor(
        audio["array"], sampling_rate=audio["sampling_rate"], padding=True
    )
    return features

minds = minds.map(prepare_dataset)
# Giờ mỗi ví dụ có thêm cột input_features (log-mel spectrogram)
```

Nhiều tác vụ âm thanh là **đa phương thức** (ví dụ nhận dạng tiếng nói cần xử lý cả văn bản). Khi đó mô hình còn có một *tokenizer* cho phần văn bản. Thay vì nạp riêng từng thứ, dùng `AutoProcessor` để lấy cả bộ trích đặc trưng lẫn tokenizer:

```
from transformers import AutoProcessor

processor = AutoProcessor.from_pretrained("openai/whisper-small")
```

Tóm tắt chương

- Âm thanh số là dãy giá trị biên độ lấy mẫu đều theo thời gian; tần số lấy mẫu và độ sâu bit quyết định độ trung thực.
- Một tín hiệu có nhiều cách nhìn: dạng sóng (thời gian), phổ (tần số), spectrogram và phổ mel (thời gian + tần số). Phổ mel bám theo thính giác người.
- Thư viện `datasets` nạp dữ liệu trong một dòng; `cast_column` đổi tần số lấy mẫu, `filter` lọc, `map` trích đặc trưng.
- Mỗi mô hình có feature extractor riêng; dùng `AutoProcessor` khi tác vụ cần cả âm thanh lẫn văn bản.

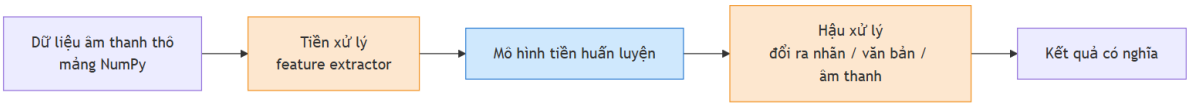
Chương 3

Ứng dụng âm thanh với `pipeline`

Trước khi mổ xẻ kiến trúc mô hình hay tinh chỉnh, hãy xem cách giải những tác vụ âm thanh phổ biến chỉ với vài dòng code, nhờ hàm `pipeline` của thư viện `transformers`. Đây là cách nhanh nhất để có một kết quả ban đầu (baseline) và cảm nhận được sức mạnh của các mô hình tiền huấn luyện.

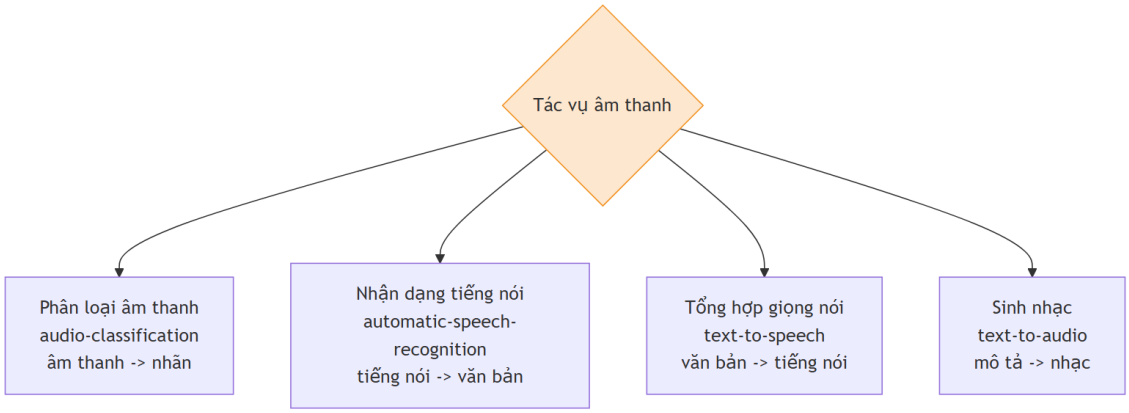
3.1 Ý tưởng của `pipeline`

Hàm `pipeline` nối mô hình với các bước tiền xử lý và hậu xử lý cần thiết. Bạn chỉ đưa dữ liệu thô vào và nhận kết quả có nghĩa ra; mọi việc chuyển đổi định dạng được lo liệu bên trong (Hình 3.1).



Hình 3.1: Ba lớp xử lý bên trong một `pipeline` âm thanh.

Cùng một hàm `pipeline` phủ được nhiều tác vụ âm thanh khác nhau, chỉ cần đổi tên tác vụ và mô hình (Hình 3.2).



Hình 3.2: Các tác vụ âm thanh thường gặp và tên `pipeline` tương ứng.

3.2 Phân loại âm thanh

Phân loại âm thanh là gán một hay nhiều nhãn cho đoạn ghi âm dựa trên nội dung: nhạc, tiếng nói, tiếng chim, tiếng động cơ… Ở đây ta phân loại các cuộc gọi MINDS-14 theo *ý định* của người gọi. Trên Hub có sẵn một mô hình đã tinh chỉnh đúng cho bộ dữ liệu này.

```
from datasets import load_dataset, Audio

minds = load_dataset("PolyAI/minds14", name="en-AU", split="train")
minds = minds.cast_column("audio", Audio(sampling_rate=16_000))
```

```
from transformers import pipeline

classifier = pipeline(
    "audio-classification",
    model="anton-l/xtreme_s_xlsr_300m_minds14",
)

example = minds[0]
classifier(example["audio"]["array"])
# [{"score": 0.9632, "label": "pay_bill"},
#  {"score": 0.0282, "label": "freeze"},
#  {"score": 0.0033, "label": "card_issues"}, ...]
```

Mô hình rất tự tin rằng người gọi muốn hỏi về việc thanh toán hoá đơn. Kiểm tra nhãn thật:

```
id2label = minds.features["intent_class"].int2str
id2label(example["intent_class"])   # -> "pay_bill" (đúng!)
```

Ở đây ta may mắn tìm được mô hình phân đúng tập nhãn ta cần. Trong thực tế, tập nhãn của mô hình tiền huấn luyện thường không trùng khớp nhu cầu; khi đó ta tinh chỉnh để "hiệu chỉnh" mô hình về đúng bộ nhãn của mình (xem Chương 5).

3.3 Nhận dạng tiếng nói

Nhận dạng tiếng nói tự động (Automatic Speech Recognition – ASR) là chép lời đoạn ghi âm thành văn bản: từ phụ đề video đến ra lệnh cho trợ lý ảo. Dùng pipeline `automatic-speech-recognition`:

```
from transformers import pipeline

asr = pipeline("automatic-speech-recognition")

example = minds[0]
asr(example["audio"]["array"])
# {"text": "I WOULD LIKE TO PAY MY ELECTRICITY BILL USING MY COD
#           CAN YOU PLEASE ASSIST"}
```

So với bản chép lời gốc ("…using my card …"), mô hình chỉ sai một từ ("card" thành "cod") – khá tốt với giọng Úc, nơi âm "r" thường bị nuốt.

Mặc định pipeline này dùng mô hình cho tiếng Anh. Muốn ngôn ngữ khác, tìm trên Hub một mô hình ASR phù hợp rồi truyền qua tham số `model`. Ví dụ với tiếng Đức:

```
minds = load_dataset("PolyAI/minds14", name="de-DE", split="train")
minds = minds.cast_column("audio", Audio(sampling_rate=16_000))
example = minds[0]

asr = pipeline("automatic-speech-recognition",
               model="maxidl/wav2vec2-large-xlsr-german")
asr(example["audio"]["array"])
# {"text": "ich möchte gerne geld auf mein konto einzallen"}
```

Vì sao nên bắt đầu bằng pipeline

- Có thể đã tồn tại một mô hình giải tốt tác vụ của bạn, tiết kiệm nhiều thời gian.
- `pipeline` lo hết tiền/hậu xử lý, bạn không phải lo định dạng dữ liệu.
- Nếu kết quả chưa lý tưởng, nó vẫn là một baseline nhanh để so sánh khi tinh chỉnh sau này.
- Khi bạn tinh chỉnh mô hình rồi chia sẻ lên Hub, cả cộng đồng dùng lại được dễ dàng qua chính `pipeline`.

3.4 Sinh âm thanh: giọng nói và nhạc

Sinh âm thanh gồm nhiều tác vụ tạo ra âm thanh ở đầu ra. Ở đây ta xem hai loại: tổng hợp giọng nói (text-to-speech) và sinh nhạc. Cả hai dùng chung pipeline `text-to-audio` (có bí danh `text-to-speech`).

3.4.1 Tổng hợp giọng nói

Biến văn bản thành giọng nói sống động, phục vụ trợ lý ảo, công cụ hỗ trợ người khiếm thị, sách nói cá nhân hoá. Ví dụ dùng mô hình Bark:

```
from transformers import pipeline

pipe = pipeline("text-to-speech", model="suno/bark-small")

text = "Ladybugs have had important roles in culture and religion, " \
       "being associated with luck, love, fertility and prophecy."
output = pipe(text)

# Trong notebook, nghe kết quả:
from IPython.display import Audio
Audio(output["audio"].squeeze(), rate=output["sampling_rate"])
```

Bark là mô hình đa ngôn ngữ: chỉ cần đổi văn bản sang tiếng Pháp là nó tự nhận ra ngôn ngữ. Nó còn tạo được giao tiếp phi ngôn ngữ và cả tiếng hát (ví dụ bọc câu hát giữa các ký hiệu nốt nhạc).

3.4.2 Sinh nhạc

Với sinh nhạc, dùng pipeline `text-to-audio` cùng mô hình MusicGen. Mô tả loại nhạc muốn tạo bằng lời, và điều khiển độ dài qua `max_new_tokens`:

```
music_pipe = pipeline("text-to-audio", model="facebook/musicgen-small")

text = "90s rock song with electric guitar and heavy drums"
output = music_pipe(text, forward_params={"max_new_tokens": 512})
Audio(output["audio"][0], rate=output["sampling_rate"])
```

Tóm tắt chương

- `pipeline` là cách nhanh nhất để thử các tác vụ âm thanh: phân loại, ASR, tổng hợp giọng nói, sinh nhạc.
- Chỉ cần đổi tên tác vụ và tên mô hình trên Hub là chuyển được giữa các tác vụ và ngôn ngữ.
- Pipeline cho một baseline tốt; muốn khớp đúng nhu cầu riêng thì tinh chỉnh, nội dung ở các chương sau.

Chương 4

Kiến trúc Transformer cho âm thanh

Chương này giải thích cách đưa âm thanh vào mô hình Transformer và ba kiểu kiến trúc hay dùng cho âm thanh: CTC (chỉ encoder), chuỗi-sang-chuỗi (encoder-decoder), và phân loại. Điểm mấu chốt cần nhớ: *các mô hình âm thanh giống nhau nhiều hơn khác nhau* – chúng dùng chung khung Transformer, chỉ thay lớp đưa dữ liệu vào/ra.

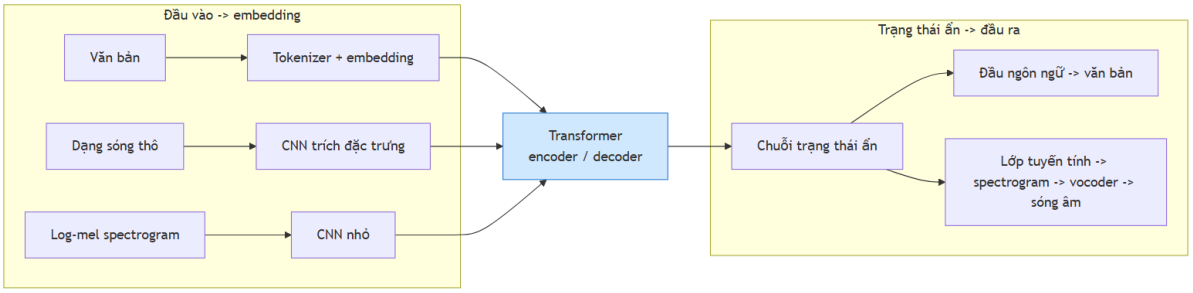
4.1 Nhắc lại Transformer trong 60 giây

Transformer gốc sinh ra cho dịch máy, gồm hai khối: **encoder** đọc đầu vào và xây biểu diễn của nó (tối ưu cho *hiểu*); **decoder** dùng biểu diễn đó cùng các token đã sinh để tạo chuỗi đích (tối ưu cho *sinh*). Có mô hình chỉ dùng encoder (hợp tác vụ hiểu, như phân loại), có mô hình chỉ dùng decoder (hợp sinh văn bản). Trái tim của Transformer là **lớp attention**: nó bảo mô hình chú ý mạnh vào vài phần tử liên quan trong chuỗi và lờ đi phần còn lại.

4.2 Đưa âm thanh vào và lấy kết quả ra

Mô hình âm thanh dùng kiến trúc Transformer tiêu chuẩn, chỉ thay đổi nhẹ ở hai đầu để nhận/trả âm thanh thay vì văn bản (Hình 4.1). Tuỳ tác vụ, đầu vào và/hoặc đầu ra có thể là âm thanh:

- **ASR**: vào là tiếng nói, ra là văn bản.
- **TTS**: vào là văn bản, ra là tiếng nói.
- **Phân loại âm thanh**: vào là âm thanh, ra là xác suất lớp.
- **Chuyển/làm sạch giọng**: cả vào lẫn ra đều là âm thanh.



Hình 4.1: Khung chung: lớp nhỏ đổi đầu vào thành embedding, Transformer xử lý, lớp khác đổi trạng thái ẩn thành đầu ra.

4.2.1 Đầu vào mô hình

Mục tiêu là biến đầu vào thành **vector embedding** mà Transformer xử lý được.

- **Đầu vào văn bản** (TTS): giống hệt mô hình NLP – văn bản được tách token rồi qua lớp embedding thành các vector, đưa vào encoder.
- **Đầu vào dạng sóng thô** (Wav2Vec2, HuBERT): dạng sóng là chuỗi số thực một chiều (biên độ theo thời gian). Nó được chuẩn hoá về trung bình 0, phương sai 1, rồi một *CNN nhỏ* (feature encoder) nén chuỗi lại, cho ra một vector embedding cho mỗi khoảng 25 ms âm thanh.
- **Đầu vào spectrogram** (Whisper): dạng sóng thô rất dài (30 giây ở 16 kHz là 480 000 giá trị). Đổi sang log-mel spectrogram cho cùng lượng thông tin nhưng nén gọn hơn nhiều, rồi một CNN nhỏ biến nó thành chuỗi embedding.

Dạng sóng hay spectrogram?

Dạng sóng thô giữ nguyên thông tin nhưng chuỗi rất dài, tốn bộ nhớ và tính toán. Spectrogram nén lại thành chuỗi ngắn hơn mà vẫn đủ thông tin, nên thường hiệu quả hơn cho mô hình dài hơi như Whisper. Dù chọn kiểu nào, luôn có một mạng nhỏ phía trước Transformer để đổi đầu vào thành embedding.

4.2.2 Đầu ra mô hình

Transformer trả ra một chuỗi **trạng thái ẩn**. Việc còn lại là biến chúng thành văn bản hoặc âm thanh:

- **Đầu ra văn bản** (ASR): gắn một "đầu" ngôn ngữ (thường là một lớp tuyến tính) rồi lấy xác suất trên các token trong từ vựng.
- **Đầu ra spectrogram** (TTS): một lớp tuyến tính chiếu trạng thái ẩn thành log-mel spectrogram; một *post-net* (các lớp tuyến tính và tích chập) làm mịn, giảm nhiễu; cuối cùng *vocoder* đổi spectrogram thành dạng sóng nghe được.

Vì mô hình thường chỉ sinh được *biên độ* của spectrogram (thiếu pha), ta cần vocoder để ước lượng pha và dựng lại sóng âm.

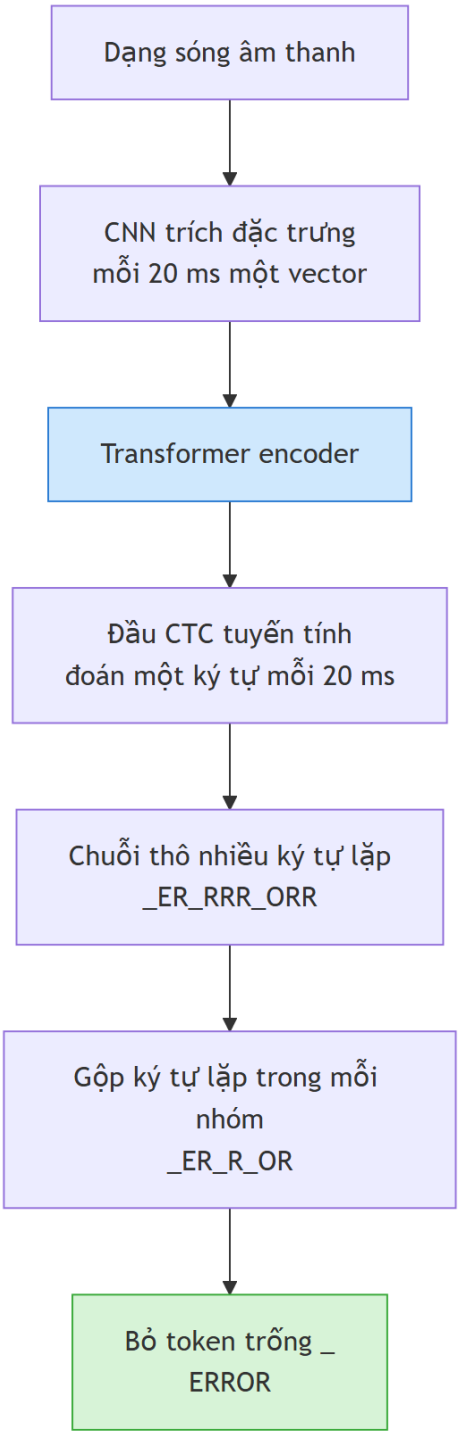
4.3 Kiến trúc 1 – CTC: ASR bằng encoder

CTC (Connectionist Temporal Classification) dùng với Transformer *chỉ encoder* cho ASR. Ví dụ: Wav2Vec2, HuBERT, M-CTC-T. Encoder đọc dạng sóng và cho ra chuỗi trạng thái ẩn; một lớp tuyến tính ("đầu CTC") đoán một *ký tự* cho mỗi trạng thái. Từ vựng chỉ cần vài chục ký tự (chữ cái + token đặc biệt) nên đầu phân loại rất nhỏ.

4.3.1 Vấn đề căn chỉnh và cách CTC giải quyết

Trong tiếng nói, ta biết thứ tự lời nói trùng thứ tự văn bản (căn chỉnh đơn điệu), nhưng *không biết* ký tự nào ứng với đoạn âm thanh nào. Dữ liệu huấn luyện chỉ có cặp (âm thanh, văn bản), không có mốc thời gian. Vì mô hình phải xuất một ký tự cho mỗi 20 ms, một ký tự kéo dài sẽ bị lặp nhiều lần trong đầu ra.

CTC giải quyết bằng một **token trống** (ký hiệu _) làm ranh giới cứng giữa các nhóm ký tự. Luật giải mã: gộp các ký tự lặp *trong cùng một nhóm*, rồi bỏ token trống (Hình 4.2).



Hình 4.2: Giải mã CTC: từ chuỗi ký tự lặp về bản chép lời đúng chính tả.

Ví dụ chuỗi thô `_ER_RRR_ORR`: gộp lặp trong từng nhóm được `_ER_R_OR`, bỏ token trống còn `ERROR`. Nếu bỏ qua token trống mà gộp thẳng `ERRRRORR` sẽ ra `EROR` – sai chính tả. Thêm CTC vào encoder rất dễ: chỉ cần một lớp tuyến tính và một hàm mất mát CTC đặc biệt.

Mẹo về từ vựng CTC và hạn chế

Từ vựng CTC nên nhỏ (dưới 50 ký tự): dùng một kiểu chữ hoa, viết số bằng chữ ("20" thành "twenty"), giữ dấu nháy ' vì "it's" khác "its". Token trống thường trùng với token đệm. Hạn chế của CTC: nó chỉ xét từng ký tự nên có thể viết sai chính tả dù nghe đúng; để cải thiện, ghép thêm một mô hình ngôn ngữ ngoài đóng vai "kiểm tra chính tả".

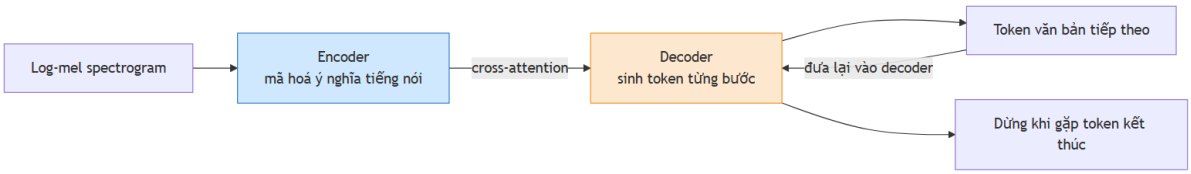
Wav2Vec2 và HuBERT dùng chung kiến trúc nhưng huấn luyện khác nhau: Wav2Vec2 học kiểu "che" giống BERT (đoán đơn vị tiếng nói ở phần bị che); HuBERT học đoán "đơn vị tiếng nói rời rạc", coi tiếng nói như token để áp các kỹ thuật NLP sẵn có.

4.4 Kiến trúc 2 – Seq2seq: encoder-decoder

Khi thêm decoder vào, ta có mô hình **chuỗi-sang-chuỗi** (seq2seq). Khác với CTC (một dự đoán cho mỗi khung, nên vào và ra dài bằng nhau), seq2seq cho phép đầu vào và đầu ra *dài khác nhau* – hợp với ASR, dịch, tóm tắt.

4.4.1 ASR với seq2seq (Whisper)

Luồng xử lý của Whisper (Hình 4.3): encoder nhận log-mel spectrogram và mã hoá "ý nghĩa" của tiếng nói thành chuỗi trạng thái ẩn. Decoder dùng cơ chế **cross-attention** (giống self-attention nhưng nhìn sang đầu ra encoder) để sinh token văn bản *từng cái một* theo kiểu tự hồi quy, bắt đầu từ token "start" và dừng khi gặp token "end".



Hình 4.3: ASR kiểu seq2seq: encoder hiểu tiếng nói, decoder sinh dần văn bản.

Decoder giống encoder nhưng có hai khác biệt: (1) có cross-attention để nhìn biểu diễn đầu vào từ encoder; (2) attention của nó *nhân quả* – không được nhìn token tương lai. Nhờ decoder đóng vai mô hình ngôn ngữ, seq2seq mạnh hơn CTC: huấn luyện đầu-cuối với cùng dữ liệu và hàm mất mát, linh hoạt và thường chính xác hơn. Whisper còn xuất token là *cả từ hoặc phần từ* (dùng tokenizer của GPT-2, hơn 50 nghìn token), nên chuỗi đầu ra ngắn hơn nhiều so với CTC.

4.4.2 TTS với seq2seq (SpeechT5)

TTS kiểu seq2seq hoạt động hệt như trên nhưng *đảo vào–ra*: encoder nhận token văn bản, decoder dùng cross-attention để sinh dần *spectrogram* từng lát thời gian một. Decoder khởi động bằng một spectrogram toàn số không (đóng vai "token start"), rồi lớn dần. Để biết khi nào dừng, SpeechT5 cho decoder đoán thêm một chuỗi xác suất "đây có phải lát cuối không"; vượt ngưỡng (ví dụ 0,5) thì dừng. Sau đó post-net làm mịn, và một vocoder (huấn luyện riêng, không thuộc seq2seq) đổi spectrogram thành sóng âm.

Vì sao TTS khó đánh giá

TTS là ánh xạ *một–nhiều*: một câu văn có thể đọc thành nhiều cách khác nhau (nhấn nhá, giọng khác nhau). Vì vậy giá trị mất mát (L1/MSE giữa các spectrogram) không phản ánh chất lượng thực. TTS thường được đánh giá bằng người nghe, qua điểm MOS (mean opinion score). Nhược điểm chung của seq2seq: giải mã từng bước nên chậm hơn, và mô hình tự hồi quy đôi khi lặp từ hoặc nhảy từ.

4.5 Kiến trúc 3 – Phân loại âm thanh

Mục tiêu là đoán nhãn lớp cho âm thanh: một nhãn cho cả đoạn (ví dụ loài chim nào đang hót), hoặc một nhãn cho mỗi khung 20 ms (ví dụ ai đang nói ở từng thời điểm – phân tách người nói).

4.5.1 Coi spectrogram như một bức ảnh

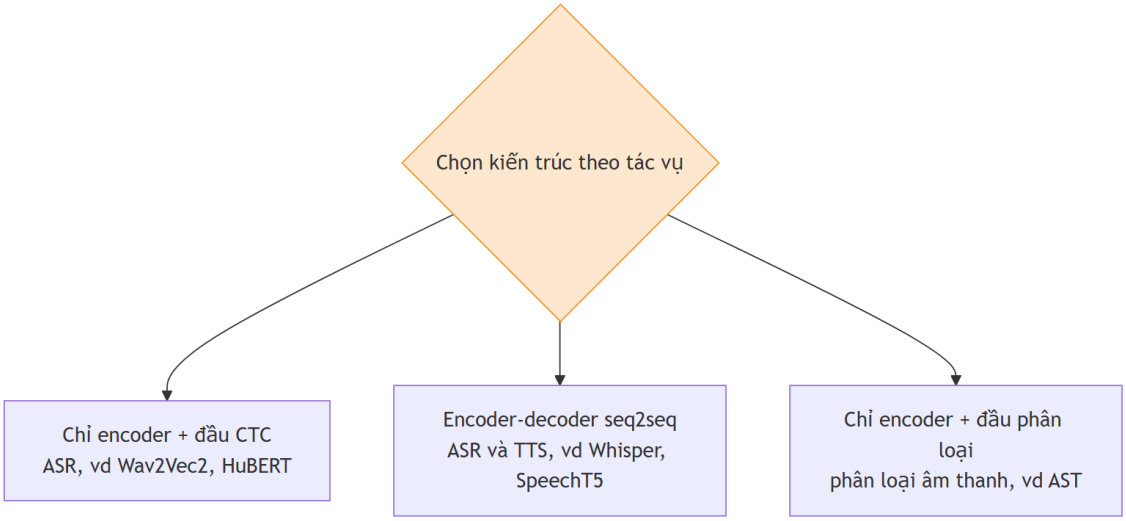
Cách dễ nhất: giả vờ đây là bài toán phân loại ảnh! Spectrogram là tensor hai chiều (tần số $\times$ thời gian), nhìn được như một bức ảnh. Vậy có thể đưa thẳng vào một CNN (như ResNet) hoặc một Transformer thị giác (ViT). Đó chính là cách **Audio Spectrogram Transformer (AST)** làm: cắt spectrogram thành các mảnh 16×16 chồng nhau một phần, chiếu thành chuỗi embedding, đưa qua encoder, rồi một lớp phân loại ở trên cho ra xác suất.

Ảnh và spectrogram không hoàn toàn giống nhau

Dịch nội dung ảnh lên/xuống thường không đổi ý nghĩa, nhưng dịch spectrogram lên/xuống sẽ đổi các tần số và thay đổi hẳn tính chất âm thanh. Ảnh bất biến với phép tịnh tiến, spectrogram thì không. Coi spectrogram như ảnh thường chạy tốt trong thực tế, nhưng hãy nhớ chúng không thật sự là một.

4.5.2 Mọi encoder đều có thể thành bộ phân loại

Bất kỳ Transformer âm thanh chỉ-encoder nào cũng thành bộ phân loại được bằng cách gắn một lớp phân loại lên chuỗi trạng thái ẩn (thường không cần decoder). Ba kiểu kiến trúc và tác vụ tương ứng được tóm ở Hình 4.4.



Hình 4.4: Ba kiểu kiến trúc Transformer cho âm thanh và tác vụ phù hợp.

Ví dụ, cùng mô hình nền Wav2Vec2 có nhiều biến thể, chỉ khác nhau ở kích thước lớp phân loại và hàm mất mát:

- `Wav2Vec2ForCTC` – cho ASR (đầu CTC).
- `Wav2Vec2ForSequenceClassification` – một nhãn cho cả đoạn: lấy trung bình các trạng thái ẩn rồi đưa vào lớp phân loại.
- `Wav2Vec2ForAudioFrameClassification` – một nhãn cho mỗi khung: chạy lớp phân loại trên từng trạng thái ẩn, cho ra một chuỗi nhãn.

Tóm tắt chương

- Mô hình âm thanh dùng chung khung Transformer; chỉ thay lớp đưa dữ liệu vào (CNN cho dạng sóng/spectrogram, embedding cho văn bản) và lớp lấy kết quả ra.
- CTC (chỉ encoder) giải ASR bằng cách đoán ký tự mỗi 20 ms và dùng token trống để gộp lặp – đơn giản, nhanh, nhưng dễ sai chính tả.
- Seq2seq (encoder-decoder) mạnh và linh hoạt hơn cho cả ASR (Whisper) và TTS (SpeechT5), đổi lại giải mã từng bước nên chậm hơn.
- Phân loại âm thanh có thể coi spectrogram như ảnh (AST) hoặc gắn đầu phân loại lên bất kỳ encoder âm thanh nào.

Chương 5

Dựng bộ phân loại thể loại nhạc

Phân loại âm thanh là một trong những ứng dụng phổ biến nhất của Transformer trong âm thanh. Chương này đi qua trọn vẹn một dự án thực tế: khảo sát các mô hình phân loại có sẵn, rồi *tinh chỉnh* một mô hình nhẹ để phân loại nhạc theo thể loại, và cuối cùng dựng demo bằng Gradio để ai cũng thử được.

5.1 Các bài toán phân loại âm thanh và mô hình có sẵn

Hub có hơn 500 mô hình phân loại âm thanh. Vì kiến trúc chuẩn muốn biến *một chuỗi âm thanh* thành *một nhãn*, người ta ưu tiên mô hình **chỉ encoder**: encoder đổi âm thanh thành chuỗi trạng thái ẩn, lấy trung bình rồi đưa qua một lớp phân loại. Mô hình chỉ-decoder hay encoder-decoder thừa phức tạp và chậm hơn nên ít dùng cho tác vụ này. Dưới đây là vài nhánh phân loại tiêu biểu.

5.1.1 Phát hiện từ khoá (keyword spotting)

Nhận ra một từ khoá trong câu nói; tập từ khoá chính là tập nhãn. Ví dụ phân loại ý định cuộc gọi MINDS-14 bằng mô hình XLS-R đã tinh chỉnh:

```
from datasets import load_dataset
from transformers import pipeline

minds = load_dataset("PolyAI/minds14", name="en-AU", split="train")
classifier = pipeline("audio-classification",
                      model="anton-l/xtreme_s_xlsr_300m_minds14")
classifier(minds[0]["audio"])
# [{"score": 0.9632, "label": "pay_bill"}, ...]  -> ý định "pay_bill", 96%
```

Một ứng dụng gần gũi là **từ đánh thức** (“Hey Siri”, “Hey Google”). Mô hình phân loại cho việc này rất nhỏ (vài triệu tham số) nên chạy liên tục trên điện thoại mà không hao pin; chỉ khi phát hiện từ đánh thức mới kích hoạt mô hình nhận dạng tiếng nói lớn hơn. Ví dụ dùng checkpoint AST tinh chỉnh trên bộ Speech Commands:

```
speech_commands = load_dataset("speech_commands", "v0.02",
                               split="validation", streaming=True)
sample = next(iter(speech_commands))

classifier = pipeline("audio-classification",
                      model="MIT/ast-finetuned-speech-commands-v2")
classifier(sample["audio"].copy())
# [{"score": 0.99999, "label": "backward"}, ...]
```

Mẹo tìm mô hình trên Hub

Vào tab *Models* của Hub, lọc theo tác vụ "Audio Classification", rồi lọc tiếp theo *Dataset* (ví dụ gõ "speech_commands"). Chỉ còn lại các mô hình đã tinh chỉnh đúng bộ dữ liệu đó. Dùng *Dataset Viewer* để nghe thử và xem metadata trước khi quyết định tải về – rất tiện để chọn đúng dữ liệu.

5.1.2 Nhận diện ngôn ngữ (LID)

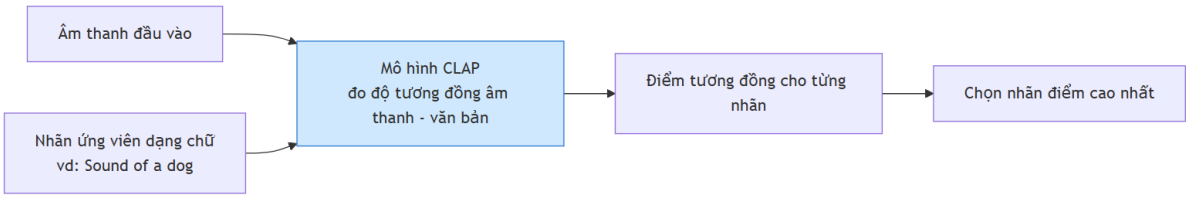
Đoán ngôn ngữ đang nói từ một danh sách ứng viên. LID hữu ích làm bước đầu trong một pipeline tiếng nói: xác định ngôn ngữ rồi chọn mô hình ASR phù hợp. Ví dụ dùng Whisper tinh chỉnh trên bộ FLEURS (102 ngôn ngữ):

```
fleurs = load_dataset("google/fleurs", "all", split="validation", streaming=True)
sample = next(iter(fleurs))

classifier = pipeline("audio-classification",
                      model="sanchit-gandhi/whisper-medium-fleurs-lang-id")
classifier(sample["audio"])
# [{"score": 0.99993, "label": "Afrikaans"}, ...]
```

5.1.3 Phân loại không mẫu (zero-shot) với CLAP

Hạn chế của phân loại truyền thống: tập nhãn của mô hình phải trùng tập nhãn tác vụ. **Zero-shot** vượt qua rào cản này. Hiện `transformers` hỗ trợ mô hình **CLAP** – nhận cả âm thanh lẫn văn bản và đo *độ tương đồng* giữa chúng (Hình 5.1). Ta đưa một âm thanh cùng nhiều nhãn dạng chữ, mô hình chấm điểm từng nhãn, chọn nhãn điểm cao nhất.



Hình 5.1: Phân loại không mẫu với CLAP: so khớp âm thanh với các nhãn dạng văn bản.

```
dataset = load_dataset("ashraq/esc50", split="train", streaming=True)
audio_sample = next(iter(dataset))["audio"]["array"]

candidate_labels = ["Sound of a dog", "Sound of vacuum cleaner"]

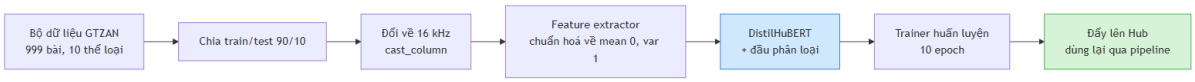
classifier = pipeline(task="zero-shot-audio-classification",
                      model="laion/clap-htsat-unfused")
classifier(audio_sample, candidate_labels=candidate_labels)
# [{"score": 0.9997, "label": "Sound of a dog"}, ...]
```

Vì sao không dùng zero-shot cho mọi việc? Vì CLAP được tiền huấn luyện trên âm thanh môi trường chung, không chuyên tiếng nói. Nó biết hai mẫu đều là tiếng nói, nhưng không phân biệt được ngôn ngữ tốt như một mô hình LID chuyên dụng.

5.2 Tinh chỉnh mô hình phân loại nhạc

Giờ ta tinh chỉnh một mô hình chỉ-encoder cho nhạc. Dùng mô hình nhẹ **DistilHuBERT** (bản "chưng cất" của HuBERT, huấn luyện nhanh hơn 73% mà giữ được phần lớn hiệu năng), chạy được trên GPU

phổ thông. Luồng tổng thể ở Hình 5.2.



Hình 5.2: Luồng tinh chỉnh DistilHuBERT để phân loại thể loại nhạc.

5.2.1 Bộ dữ liệu GTZAN

GTZAN gồm 1 000 bài, mỗi bài một đoạn 30 giây thuộc một trong 10 thể loại (từ disco đến metal). Một bản ghi hỏng đã bị loại, nên còn 999 ví dụ. GTZAN không có sẵn tập kiểm định nên ta tự chia 90/10:

```
from datasets import load_dataset

gtzan = load_dataset("marsyas/gtzan", "all")       # 999 dòng
gtzan = gtzan["train"].train_test_split(seed=42, shuffle=True, test_size=0.1)
# train: 899, test: 100

# Nhãn là số; đổi sang tên thể loại cho dễ đọc
id2label_fn = gtzan["train"].features["genre"].int2str
id2label_fn(gtzan["train"][0]["genre"])            # -> "pop"
```

Các bài trong GTZAN lấy mẫu ở 22 050 Hz, trong khi DistilHuBERT cần 16 000 Hz.

5.2.2 Tiền xử lý: trích đặc trưng và chuẩn hoá

HuBERT và Wav2Vec2 nhận *dạng sóng thô* (không phải spectrogram như Whisper). Dữ liệu âm thanh đã đúng định dạng mảng số, vậy feature extractor làm gì? Nó **chuẩn hoá** – đưa mỗi mẫu về trung bình 0, phương sai 1 – để mọi đầu vào cùng một dải động, giúp huấn luyện ổn định và hội tụ tốt hơn.

```
from transformers import AutoFeatureExtractor
from datasets import Audio

model_id = "ntu-spml/distilhubert"
feature_extractor = AutoFeatureExtractor.from_pretrained(
    model_id, do_normalize=True, return_attention_mask=True
)

# Đổi tần số lấy mẫu về đúng mức mô hình cần (16 kHz)
sampling_rate = feature_extractor.sampling_rate      # 16000
gtzan = gtzan.cast_column("audio", Audio(sampling_rate=sampling_rate))
```

Thử áp dụng feature extractor cho một mẫu: nó trả về từ điển gồm `input_values` (âm thanh đã chuẩn hoá) và `attention_mask` (đánh dấu phần đệm khi xử lý theo lô). Sau chuẩn hoá, trung bình gần 0 và phương sai đúng bằng 1.

```
import numpy as np

sample = gtzan["train"][0]["audio"]
inputs = feature_extractor(sample["array"], sampling_rate=sample["sampling_rate"])
# inputs keys: ['input_values', 'attention_mask']
# Mean: -4.53e-09, Variance: 1.0
```

Vì các đoạn dài 30 giây, ta cắt bớt đoạn dài hơn rồi áp hàm tiền xử lý cho cả bộ dữ liệu bằng `map` theo lô:

```
max_duration = 30.0

def preprocess_function(examples):
    audio_arrays = [x["array"] for x in examples["audio"]]
    inputs = feature_extractor(
        audio_arrays,
        sampling_rate=feature_extractor.sampling_rate,
        max_length=int(feature_extractor.sampling_rate * max_duration),
        truncation=True,
        return_attention_mask=True,
    )
    return inputs

gtzan_encoded = gtzan.map(
    preprocess_function, remove_columns=["audio", "file"],
    batched=True, batch_size=100, num_proc=1,
)
# Đổi tên cột "genre" thành "label" để Trainer hiểu
gtzan_encoded = gtzan_encoded.rename_column("genre", "label")
```

Hết RAM thì làm sao

Nếu máy hết RAM khi chạy `map`, giảm `batch_size` (1000 xuống 100 xuống 50) hoặc `writer_batch_size` (1000 xuống 500 xuống 250). Khi huấn luyện gặp lỗi CUDA "out-of-memory", giảm `batch_size` theo bội số 2 và bù lại bằng `gradient_accumulation_steps`.

Cuối cùng, lập ánh xạ hai chiều giữa số và tên nhãn để diễn giải dự đoán của mô hình:

```
id2label = {str(i): id2label_fn(i)
            for i in range(len(gtzan_encoded["train"].features["label"].names))}
label2id = {v: k for k, v in id2label.items()}
id2label["7"]   # -> "pop"
```

5.2.3 Huấn luyện với `Trainer`

Dùng lớp `Trainer` – API cấp cao lo giúp phần lớn tình huống huấn luyện. Trước hết nạp mô hình với đầu phân loại phù hợp qua `AutoModelForAudioClassification`:

```
from transformers import AutoModelForAudioClassification

model = AutoModelForAudioClassification.from_pretrained(
    model_id, num_labels=len(id2label),
    label2id=label2id, id2label=id2label,
)
```

Định nghĩa tham số huấn luyện (cỡ lô, số epoch, tốc độ học…). Đặt `push_to_hub=True` để tự đẩy checkpoint lên Hub trong lúc huấn luyện:

```
from transformers import TrainingArguments

model_name = model_id.split("/")[-1]

training_args = TrainingArguments(
    f"{model_name}-finetuned-gtzan",
    eval_strategy="epoch", save_strategy="epoch",
```

```
    learning_rate=5e-5,
    per_device_train_batch_size=8, per_device_eval_batch_size=8,
    gradient_accumulation_steps=1, num_train_epochs=10,
    warmup_steps=100, logging_steps=5,
    load_best_model_at_end=True, metric_for_best_model="accuracy",
    fp16=True, push_to_hub=True,
)
```

Vì bộ dữ liệu cân bằng, dùng *độ chính xác* (accuracy) làm thước đo, nạp qua thư viện Evaluate; rồi khởi tạo `Trainer` và huấn luyện:

```
import evaluate
import numpy as np

metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    predictions = np.argmax(eval_pred.predictions, axis=1)
    return metric.compute(predictions=predictions,
                          references=eval_pred.label_ids)

from transformers import Trainer

trainer = Trainer(
    model, training_args,
    train_dataset=gtzan_encoded["train"],
    eval_dataset=gtzan_encoded["test"],
    tokenizer=feature_extractor,
    compute_metrics=compute_metrics,
)
trainer.train()
```

Huấn luyện 10 epoch mất khoảng 1 giờ tuỳ GPU. Theo bảng kết quả trong tài liệu gốc, độ chính xác kiểm định tốt nhất đạt **83%** – khá tốt với chỉ 899 ví dụ. Có thể cải thiện bằng cách huấn luyện thêm epoch, dùng dropout, hoặc chia mỗi đoạn 30 giây thành hai đoạn 15 giây để tăng dữ liệu. Xong thì đẩy kết quả lên Hub:

```
kwargs = {
    "dataset_tags": "marsyas/gtzan", "dataset": "GTZAN",
    "model_name": f"{model_name}-finetuned-gtzan",
    "finetuned_from": model_id, "tasks": "audio-classification",
}
trainer.push_to_hub(**kwargs)
```

Vì sao nên đẩy lên Hub

Hub cho bạn quản lý phiên bản (không mất checkpoint), nhật ký Tensorboard để theo dõi chỉ số, thẻ mô hình (model card) để ghi lại công dụng, và dễ chia sẻ với cộng đồng. Quy trình này áp dụng được cho *mọi* tác vụ phân loại âm thanh – chỉ cần đổi bộ dữ liệu, ví dụ sang phát hiện từ khoá hay nhận diện ngôn ngữ.

5.3 Dựng demo với Gradio

Cuối cùng, dựng một demo để thử mô hình vừa tinh chỉnh. Nạp checkpoint qua `pipeline`, viết hàm nhận đường dẫn file âm thanh và trả về xác suất từng nhãn:

```
from transformers import pipeline
import gradio as gr

model_id = "sanchit-gandhi/distilhubert-finetuned-gtzan"
pipe = pipeline("audio-classification", model=model_id)

def classify_audio(filepath):
    preds = pipe(filepath)        # pipeline tự nạp file, đổi tần số, chạy mô hình
    return {p["label"]: p["score"] for p in preds}

demo = gr.Interface(
    fn=classify_audio,
    inputs=gr.Audio(type="filepath"),
    outputs=gr.Label(),
)
demo.launch(debug=True)
```

Tóm tắt chương

- Phân loại âm thanh ưu tiên mô hình chỉ-encoder; nhiều nhánh: phát hiện từ khoá, nhận diện ngôn ngữ, zero-shot (CLAP).
- Dùng bộ lọc trên Hub (theo tác vụ và dataset) để tìm nhanh mô hình phù hợp.
- Tinh chỉnh gồm: nạp dữ liệu, đổi tần số lấy mẫu, trích đặc trưng (chuẩn hoá), huấn luyện bằng `Trainer`, rồi đẩy lên Hub.
- Vài dòng Gradio là đủ dựng demo và chia sẻ.

Chương 6

Nhận dạng tiếng nói tự động

Chương này đi sâu vào nhận dạng tiếng nói (ASR): các họ mô hình và điểm mạnh/yếu, cách chọn bộ dữ liệu, cách đo chất lượng bằng tỉ lệ lỗi từ (WER), và một ví dụ tinh chỉnh Whisper trọn vẹn cho một ngôn ngữ ít tài nguyên.

6.1 Hai họ mô hình ASR

Như đã nói ở Chương 4, mô hình ASR chia làm hai họ: CTC (chỉ encoder + đầu phân loại tuyến tính) và seq2seq (encoder-decoder với cross-attention). Trước 2022, CTC phổ biến hơn: Wav2Vec2, HuBERT, XLSR được tiền huấn luyện trên khối âm thanh *không nhãn* khổng lồ, rồi người dùng chỉ cần vài phút dữ liệu có nhãn để tinh chỉnh.

6.1.1 Mổ xẻ mô hình CTC (Wav2Vec2)

Thử Wav2Vec2 trên một mẫu LibriSpeech:

```
from datasets import load_dataset
from transformers import pipeline

dataset = load_dataset("hf-internal-testing/librispeech_asr_dummy",
                       "clean", split="validation")
sample = dataset[2]

pipe = pipeline("automatic-speech-recognition", model="facebook/wav2vec2-base-100h")
pipe(sample["audio"].copy())
# {"text": "... WITH CHRISTMAUS AND ROSE BEEF LOOMING ... SIMALYIS DRAWN ..."}
```

Mọi từ *nghe* đúng nhưng vài từ sai chính tả: `CHRISTMAUS` (đúng là `CHRISTMAS`), `ROSE` (đúng là `ROAST`), `SIMALYIS` (đúng là `SIMILES`). Đây là điểm yếu của CTC: nó gần như là mô hình "chỉ âm học" – dựa chủ yếu vào âm thanh nên hay viết theo kiểu phiên âm, ít dùng ngữ cảnh ngôn ngữ. Ngoài ra kết quả thiếu cả chữ hoa lẫn dấu câu, hạn chế ứng dụng thực tế.

6.1.2 Bước tiến seq2seq: Whisper

Seq2seq khắc phục điều đó: decoder đóng vai mô hình ngôn ngữ, nhìn toàn bộ ngữ cảnh nên sửa được lỗi chính tả. Đổi lại, seq2seq giải mã từng bước (chậm hơn) và "ngốn" nhiều dữ liệu hơn. Whisper (OpenAI, 9/2022) giải bài toán dữ liệu bằng cách tiền huấn luyện trên 680 000 giờ âm thanh *có nhãn* (so với 60 000 giờ không nhãn của Wav2Vec2), trong đó 117 000 giờ là đa ngôn ngữ – phủ hơn 96 ngôn ngữ, nhiều ngôn ngữ ít tài nguyên.

```
import torch
from transformers import pipeline

device = "cuda:0" if torch.cuda.is_available() else "cpu"
pipe = pipeline("automatic-speech-recognition",
                model="openai/whisper-base", device=device)

pipe(sample["audio"], max_new_tokens=256)
# {"text": " He tells us that at this festive season of the year,
#            with Christmas and roast beef ..."}
```

Whisper sửa đúng `Christmas` và `roast`, lại có cả chữ hoa và dấu câu nên dễ đọc hơn hẳn. Nó có năm cỡ (tiny, base, small, medium, large); cỡ càng lớn càng chính xác nhưng cần nhiều VRAM và chạy chậm hơn. Bảng cỡ mô hình trong tài liệu gốc giúp chọn checkpoint hợp phần cứng.

6.1.3 Dịch giọng nói và đa ngôn ngữ

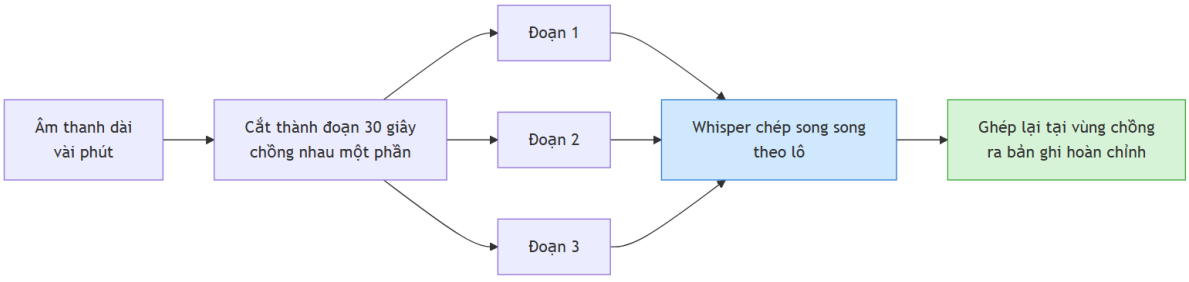
Whisper làm được cả hai tác vụ qua tham số `task`: `"transcribe"` (chép lời cùng ngôn ngữ) hoặc `"translate"` (dịch sang tiếng Anh).

```
# Chép lời: tiếng Tây Ban Nha -> văn bản Tây Ban Nha
pipe(sample["audio"].copy(), max_new_tokens=256,
     generate_kwargs={"task": "transcribe"})

# Dịch: tiếng Tây Ban Nha -> văn bản tiếng Anh
pipe(sample["audio"], max_new_tokens=256,
     generate_kwargs={"task": "translate"})
```

6.1.4 Chép âm thanh dài và dấu thời gian

Whisper vốn chỉ xử lý đoạn 30 giây. Với âm thanh dài, `transformers` dùng kỹ thuật **cắt đoạn** (chunking): chia thành các đoạn nhỏ chồng nhau một phần, chép song song rồi ghép lại tại vùng chồng (Hình 6.1). Vì thuật toán không phụ thuộc trạng thái giữa các đoạn, có thể gộp lô chạy song song.



Hình 6.1: Chép âm thanh dài bằng cách cắt đoạn 30 giây chồng nhau rồi ghép lại.

```
pipe(long_audio, max_new_tokens=256,
     generate_kwargs={"task": "transcribe"},
     chunk_length_s=30,       # 30 giây là mức tối ưu cho Whisper
     batch_size=8,            # gộp lô để chạy nhanh hơn
     return_timestamps=True)  # trả thêm dấu thời gian từng đoạn
# Mỗi phần tử: {"timestamp": (0.0, 26.4), "text": "..."}
```

Dấu thời gian rất hữu ích khi làm phụ đề: biết đoạn văn bản nào ứng với khoảng thời gian nào của video.

6.2 Chọn bộ dữ liệu

Mô hình chỉ tốt bằng dữ liệu huấn luyện nó. Khi chọn bộ dữ liệu ASR, cân nhắc bốn đặc điểm:

- **Số giờ**: quy mô dữ liệu. Nhưng lớn chưa chắc tốt – quan trọng là *đa dạng* (nhiều người nói, nhiều phong cách, nhiều miền).
- **Miền (domain)**: nguồn dữ liệu (sách nói, podcast, YouTube, họp tài chính…). Phải khớp miền với điều kiện lúc suy luận: huấn luyện trên sách nói thu âm phòng thu thì đừng mong chạy tốt trong môi trường ồn.
- **Phong cách nói**: *đọc theo kịch bản* (rõ ràng, ít lỗi) hay *nói tự nhiên* (có lặp, ngập ngừng, nói hụt).
- **Phong cách chép lời**: có chữ hoa và dấu câu hay không. Muốn văn bản định dạng đầy đủ (cho họp, xuất bản) thì cần dữ liệu có chữ hoa + dấu câu.

Hub có hơn 180 bộ dữ liệu ASR. Bảng 6.1 tóm vài bộ tiếng Anh phổ biến để làm điểm tham chiếu khi chọn.

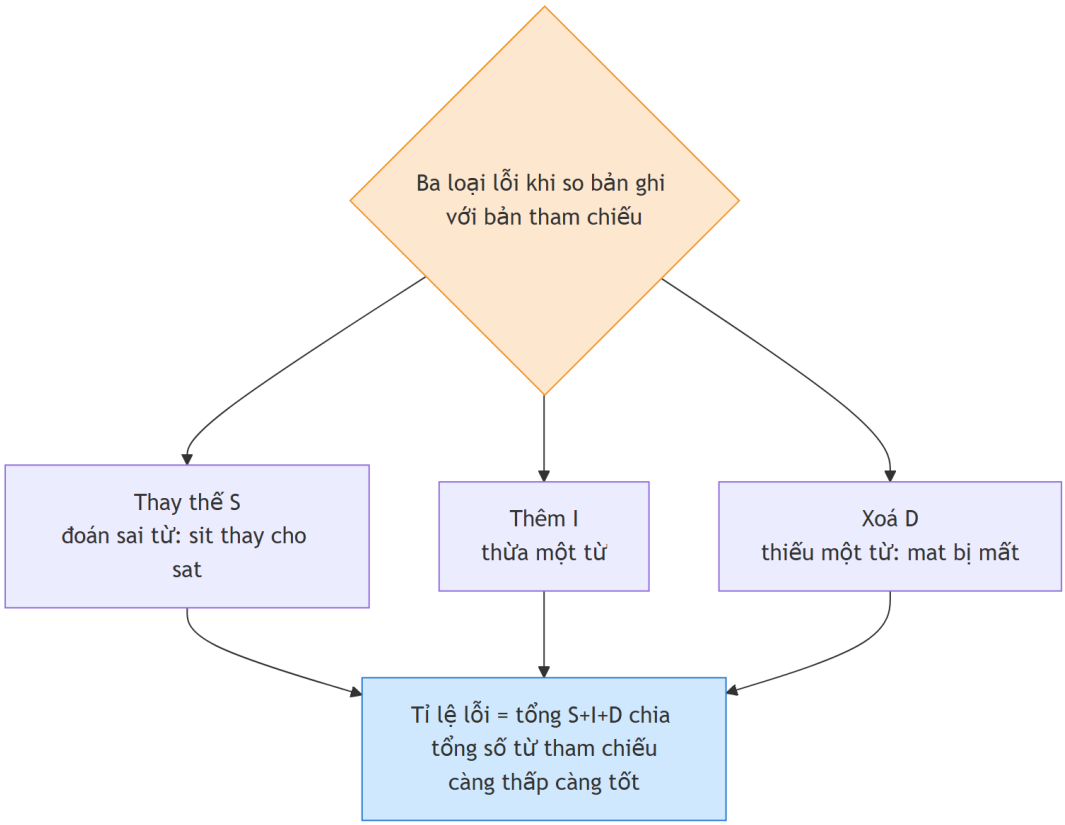
Bộ dữ liệu	Giờ	Miền / phong cách	Hoa+Dấu
LibriSpeech	960	Sách nói, đọc theo kịch bản	Không
Common Voice 11	3000	Văn Wikipedia, đọc	Có
VoxPopuli	540	Nghị viện châu Âu, diễn thuyết	Chỉ dấu câu
TED-LIUM	450	Bài nói TED, diễn thuyết	Không
GigaSpeech	10000	Sách nói/podcast/YouTube, hỗn hợp	Chỉ dấu câu
SPGISpeech	5000	Họp tài chính, hỗn hợp	Có
AMI	100	Họp, nói tự nhiên, nhiều nhiễu	Có

Bảng 6.1: Một số bộ dữ liệu ASR tiếng Anh phổ biến trên Hub.

Cho phần tinh chỉnh dưới đây, vì Whisper đã giỏi tiếng Anh/Tây Ban Nha, ta chọn một ngôn ngữ ít tài nguyên để thấy rõ tác dụng: **Dhivehi** (Maldives) trong bộ **Common Voice 13** – bộ chúng ta muốn vì vẫn giữ được chữ hoa và dấu câu.

6.3 Đo chất lượng: tỉ lệ lỗi từ (WER)

Để đánh giá ASR, ta so bản ghi của hệ thống với bản chép tham chiếu và đếm ba loại lỗi (Hình 6.2): **thay thế** (sai từ), **thêm** (thừa từ), **xoá** (thiếu từ).



Hình 6.2: Ba loại lỗi và cách tính tỉ lệ lỗi.

Tỉ lệ lỗi từ (Word Error Rate – WER) là thước đo chuẩn: lấy tổng số lỗi (thay + thêm + xoá) chia cho tổng số từ trong bản tham chiếu. Với ví dụ tham chiếu "the cat sat on the mat" và dự đoán "the cat sit on the": có 1 thay thế (sit thay sat) và 1 xoá (thiếu mat), tổng 2 lỗi trên 6 từ, WER $\approx 0{,}333$ (33,3%). WER *càng thấp càng tốt*, bằng 0 là hoàn hảo. Lưu ý: WER *không có trần trên* – nếu đoán 10 từ sai hết mà tham chiếu chỉ 2 từ thì WER tới 500%.

```
pip install --upgrade evaluate jiwer
```

```
from evaluate import load

reference = "the cat sat on the mat"
prediction = "the cat sit on the"

wer_metric = load("wer")
wer_metric.compute(references=[reference], predictions=[prediction])
# 0.3333...
```

Tỉ lệ lỗi ký tự (CER) tính ở mức ký tự thay vì từ: với "sit" thay "sat", chỉ chữ "i" bị tính sai nên phạt nhẹ hơn. Thường ta dùng WER vì nó buộc hệ thống hiểu ngữ cảnh (chia đúng thì của động từ). Nhưng với ngôn ngữ không có khái niệm "từ" như tiếng Trung, tiếng Nhật, ta quay về dùng CER. Còn **nghịch đảo hệ số thời gian thực** (RTFx) đo *tốc độ*: thời lượng âm thanh chia thời gian xử lý; lớn hơn 1 nghĩa là chép nhanh hơn thời gian thực. Khi đánh giá, nên nhìn cả WER lẫn RTFx.

Chuẩn hoá văn bản khi đo

Huấn luyện trên văn bản có chữ hoa và dấu câu (dạng *orthographic*) cho kết quả đẹp, dễ đọc. Nhưng khi tính WER, nếu *chuẩn hoá* (bỏ hoa và dấu câu) ở cả bản ghi lẫn tham chiếu thì WER giảm hẳn vì bài toán dễ hơn. Khuyến nghị: **huấn luyện trên văn bản đầy đủ, đánh giá trên văn bản đã chuẩn hoá** để được cả hai cái lợi. Whisper có sẵn bộ chuẩn hoá `BasicTextNormalizer`.

Theo tài liệu gốc, khi đánh giá Whisper small trên tập test Dhivehi của Common Voice 13 (làm baseline trước khi tinh chỉnh), WER dạng orthographic khoảng 167%, còn WER sau chuẩn hoá khoảng 126% – nghĩa là mô hình gốc gần như "nói nhảm" với Dhivehi. Đây chính là con số ta sẽ cố vượt qua bằng tinh chỉnh.

6.4 Tinh chỉnh Whisper cho một ngôn ngữ mới

Giờ ta tinh chỉnh Whisper small cho Dhivehi. Luồng tổng thể ở Hình 6.3.



Hình 6.3: Luồng tinh chỉnh Whisper cho tiếng Dhivehi trên Common Voice 13.

6.4.1 Nạp dữ liệu và processor

Dhivehi có khoảng 10 giờ dữ liệu có nhãn (3 giờ để test). Vì rất ít, ta gộp `train`+`validation` (7 giờ) để huấn luyện, giữ lại cả vốn đa ngôn ngữ Whisper đã học:

```
from datasets import load_dataset, DatasetDict

common_voice = DatasetDict()
common_voice["train"] = load_dataset(
    "mozilla-foundation/common_voice_13_0", "dv", split="train+validation")
common_voice["test"] = load_dataset(
    "mozilla-foundation/common_voice_13_0", "dv", split="test")

# Chỉ giữ audio và câu chép lời, bỏ metadata
common_voice = common_voice.select_columns(["audio", "sentence"])
```

Pipeline ASR gồm ba phần: *feature extractor* (âm thanh sang log-mel), *mô hình* (ánh xạ chuỗi-sang-chuỗi), và *tokenizer* (token sang văn bản). Whisper gói cả feature extractor lẫn tokenizer vào một `WhisperProcessor`. Dhivehi không nằm trong 96 ngôn ngữ Whisper biết, nhưng nó gần với tiếng Sinhala (có trong danh sách), nên ta đặt `language="sinhalese"` để tận dụng tri thức chéo ngôn ngữ:

```
from transformers import WhisperProcessor

processor = WhisperProcessor.from_pretrained(
    "openai/whisper-small", language="sinhalese", task="transcribe")
```

Khi ngôn ngữ của bạn đã được Whisper biết

Phần lớn trường hợp, ngôn ngữ bạn cần nằm sẵn trong tập 96 ngôn ngữ, chỉ việc đặt `language` đúng tên ngôn ngữ nguồn. Với tinh chỉnh chỉ-tiếng-Anh thì bỏ cả hai tham số `language` và `task`.

6.4.2 Tiền xử lý

Common Voice ở 48 kHz, cần giảm về 16 kHz. Rồi dùng processor để tính log-mel và mã hoá nhãn, cuối cùng lọc các đoạn dài hơn 30 giây:

```
from datasets import Audio

sampling_rate = processor.feature_extractor.sampling_rate   # 16000
common_voice = common_voice.cast_column("audio", Audio(sampling_rate=sampling_rate))

def prepare_dataset(example):
    audio = example["audio"]
    example = processor(audio=audio["array"],
                        sampling_rate=audio["sampling_rate"],
                        text=example["sentence"])
    example["input_length"] = len(audio["array"]) / audio["sampling_rate"]
    return example

common_voice = common_voice.map(
    prepare_dataset, remove_columns=common_voice.column_names["train"], num_proc=1)

def is_audio_in_length_range(length):
    return length < 30.0

common_voice["train"] = common_voice["train"].filter(
    is_audio_in_length_range, input_columns=["input_length"])
```

6.4.3 Collator, thước đo và cấu hình huấn luyện

Mô hình seq2seq cần một **data collator** xử lý `input_features` (đã cố định kích thước, chỉ cần đóng gói thành tensor) và `labels` (đệm tới độ dài lớn nhất trong lô, thay token đệm bằng -100 để không tính vào mất mát) riêng:

```
import torch
from dataclasses import dataclass
from typing import Any, Dict, List, Union

@dataclass
class DataCollatorSpeechSeq2SeqWithPadding:
    processor: Any
    def __call__(self, features: List[Dict[str, Union[List[int], torch.Tensor]]]):
        input_features = [{"input_features": f["input_features"][0]} for f in features]
        batch = self.processor.feature_extractor.pad(input_features, return_tensors="pt")

        label_features = [{"input_ids": f["labels"]} for f in features]
        labels_batch = self.processor.tokenizer.pad(label_features, return_tensors="pt")
        labels = labels_batch["input_ids"].masked_fill(
            labels_batch.attention_mask.ne(1), -100)
        if (labels[:, 0] == self.processor.tokenizer.bos_token_id).all().cpu().item():
            labels = labels[:, 1:]
        batch["labels"] = labels
        return batch

data_collator = DataCollatorSpeechSeq2SeqWithPadding(processor=processor)
```

Thước đo là WER (nên tính cả bản orthographic lẫn bản chuẩn hoá). Nạp mô hình, tắt cache (vì dùng gradient checkpointing), ép ngôn ngữ/tác vụ khi sinh:

```
import evaluate
from transformers.models.whisper.english_normalizer import BasicTextNormalizer
from transformers import WhisperForConditionalGeneration
from functools import partial

metric = evaluate.load("wer")
normalizer = BasicTextNormalizer()

def compute_metrics(pred):
    pred_ids, label_ids = pred.predictions, pred.label_ids
    label_ids[label_ids == -100] = processor.tokenizer.pad_token_id
    pred_str = processor.batch_decode(pred_ids, skip_special_tokens=True)
    label_str = processor.batch_decode(label_ids, skip_special_tokens=True)
    wer_ortho = 100 * metric.compute(predictions=pred_str, references=label_str)
    pred_norm = [normalizer(p) for p in pred_str]
    ref_norm = [normalizer(l) for l in label_str]
    pred_norm = [pred_norm[i] for i in range(len(pred_norm)) if len(ref_norm[i]) > 0]
    ref_norm = [ref_norm[i] for i in range(len(ref_norm)) if len(ref_norm[i]) > 0]
    wer = 100 * metric.compute(predictions=pred_norm, references=ref_norm)
    return {"wer_ortho": wer_ortho, "wer": wer}

model = WhisperForConditionalGeneration.from_pretrained("openai/whisper-small")
model.config.use_cache = False
model.generate = partial(model.generate, language="sinhalese",
                         task="transcribe", use_cache=True)
```

Cấu hình huấn luyện: đặt 500 bước (đủ thấy WER cải thiện mạnh, chạy 45 phút trên Colab free):

```
from transformers import Seq2SeqTrainingArguments, Seq2SeqTrainer

training_args = Seq2SeqTrainingArguments(
    output_dir="./whisper-small-dv",
    per_device_train_batch_size=16, gradient_accumulation_steps=1,
    learning_rate=1e-5, lr_scheduler_type="constant_with_warmup",
    warmup_steps=50, max_steps=500,       # tăng lên 4000 nếu có GPU riêng
    gradient_checkpointing=True, fp16=True, fp16_full_eval=True,
    evaluation_strategy="steps", per_device_eval_batch_size=16,
    predict_with_generate=True, generation_max_length=225,
    save_steps=500, eval_steps=500, logging_steps=25,
    load_best_model_at_end=True, metric_for_best_model="wer",
    greater_is_better=False, push_to_hub=True,
)

trainer = Seq2SeqTrainer(
    args=training_args, model=model,
    train_dataset=common_voice["train"], eval_dataset=common_voice["test"],
    data_collator=data_collator, compute_metrics=compute_metrics,
    tokenizer=processor,
)
trainer.train()
```

Theo tài liệu gốc, sau 500 bước (45 phút), WER chuẩn hoá đạt **14,1%** – cải thiện khoảng 112% so với mô hình gốc, với chỉ 7 giờ dữ liệu. Có thể tăng `max_steps` lên 4000 (và đổi lịch tốc độ học sang kiểu tuyến tính, `lr_scheduler_type="linear"`) để giảm WER thêm. Xong thì đẩy lên Hub:

```
trainer.push_to_hub(**{
    "dataset": "Common Voice 13", "language": "dv",
```

```
    "model_name": "Whisper Small Dv", "finetuned_from": "openai/whisper-small",
    "tasks": "automatic-speech-recognition",
})
```

6.5 Dựng demo

Nạp checkpoint đã tinh chỉnh qua `pipeline` và dùng y như mô hình tiền huấn luyện:

```
from transformers import pipeline

pipe = pipeline("automatic-speech-recognition",
                model="sanchit-gandhi/whisper-small-dv")
```

Có thể dựng giao diện Gradio tương tự Chương 5, nhận âm thanh từ micro hoặc file và trả về văn bản chép lời.

Tóm tắt chương

- CTC (Wav2Vec2) nhanh, nhẹ nhưng hay sai chính tả và thiếu dấu câu; seq2seq (Whisper) chính xác hơn, có hoa + dấu câu, làm được cả dịch giọng nói.
- Chọn bộ dữ liệu theo số giờ, miền, phong cách nói và phong cách chép lời; khớp với điều kiện lúc suy luận.
- WER là thước đo chuẩn (càng thấp càng tốt, không có trần); dùng CER cho ngôn ngữ không có khái niệm từ; cân nhắc cả RTFx về tốc độ.
- Tinh chỉnh Whisper: nạp dữ liệu, đổi 16 kHz, processor, collator, đo WER, huấn luyện bằng `Seq2SeqTrainer`, rồi chia sẻ qua Hub.

Chương 7

Từ văn bản sang tiếng nói

Chương này bàn về tổng hợp giọng nói (TTS): vì sao TTS khó, các bộ dữ liệu phù hợp, ba mô hình tiền huấn luyện sẵn dùng (SpeechT5, Bark, MMS), và cách đánh giá chất lượng.

7.1 Vì sao TTS khó

TTS (còn gọi là tổng hợp tiếng nói) có vài thách thức riêng:

- **Ánh xạ một–nhiều**: cùng một câu văn có thể đọc theo vô số cách (giọng, nhịp, nhấn nhá khác nhau) mà cách nào cũng đúng. Mô hình phải học thời lượng và nhịp điệu hợp lý cho từng âm, từng từ.
- **Phụ thuộc xa**: nghĩa một câu phụ thuộc ngữ cảnh các từ xung quanh; mô hình phải giữ ngữ cảnh qua chuỗi dài để nói trôi chảy, tự nhiên.
- **Dữ liệu đắt**: cần cặp (văn bản, âm thanh) sạch, đa dạng người nói. Dữ liệu ASR thường *không* hợp vì có nhiều nhiễu nền (tốt cho ASR nhưng dở cho TTS – bạn không muốn trợ lý trả lời kèm tiếng còi xe).

7.2 Bộ dữ liệu cho TTS

Một bộ dữ liệu TTS tốt cần: thu âm chất lượng cao, đa dạng (giọng, phong cách, ngôn ngữ, cảm xúc), sạch nhiễu; mỗi đoạn âm thanh kèm bản chép lời; nội dung ngôn ngữ phong phú. Vài bộ phổ biến trên Hub:

Bộ dữ liệu	Đặc điểm
LJSpeech	13 100 đoạn tiếng Anh, một người nói đọc 7 cuốn phi hư cấu; chất lượng cao, hay dùng làm chuẩn so sánh.
Multilingual LibriSpeech	Mở rộng đa ngôn ngữ của LibriSpeech (Đức, Hà Lan, Tây Ban Nha, Pháp, Ý…); hợp TTS đa ngôn ngữ.
VCTK	110 người nói tiếng Anh nhiều giọng khác nhau; hợp để tạo giọng đa dạng.
LibriTTS / LibriTTS-R	585 giờ tiếng Anh nhiều người nói ở 24 kHz; cắt theo câu, loại đoạn nhiều nhiễu, giữ cả văn bản gốc lẫn chuẩn hoá.

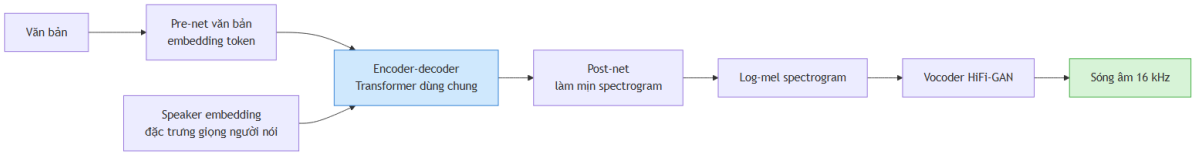
Bảng 7.1: Một số bộ dữ liệu TTS trên Hub.

Tin tốt: hiếm khi phải huấn luyện TTS từ đầu – thường chỉ cần dùng hoặc tinh chỉnh mô hình có sẵn.

7.3 SpeechT5: encoder-decoder + vocoder

SpeechT5 (Microsoft) xử lý được nhiều tác vụ tiếng nói. Lõi của nó là một Transformer encoder-decoder tiêu chuẩn, bọc thêm sáu *pre-net* và *post-net* chuyên biệt cho từng phương thức (tiếng nói/văn bản). Đầu vào đi qua pre-net phù hợp để thành biểu diễn ẩn; đầu ra Transformer qua post-net để tạo kết quả. SpeechT5 được tiền huấn luyện chung trên khối tiếng nói + văn bản không nhãn, rồi tinh chỉnh riêng cho từng tác vụ (chỉ dùng pre-net/post-net liên quan).

Với TTS, luồng xử lý ở Hình 7.1: pre-net văn bản (embedding token), Transformer dùng chung, post-net làm mịn spectrogram, cuối cùng vocoder đổi log-mel spectrogram thành sóng âm.



Hình 7.1: Luồng TTS của SpeechT5: từ văn bản tới sóng âm, qua vocoder HiFi-GAN.

```
from transformers import SpeechT5Processor, SpeechT5ForTextToSpeech

processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_tts")
model = SpeechT5ForTextToSpeech.from_pretrained("microsoft/speecht5_tts")

inputs = processor(text="Don't count the days, make the days count.",
                   return_tensors="pt")
```

SpeechT5 không bó buộc một giọng duy nhất: nó dùng **speaker embedding** – một vector cố định tóm tắt đặc trưng giọng (chất giọng, chất âm, ngữ điệu). Ta nạp sẵn một embedding kiểu X-Vector từ bộ CMU ARCTIC:

```
from datasets import load_dataset
import torch

embeddings_dataset = load_dataset("Matthijs/cmu-arctic-xvectors", split="validation")
speaker_embeddings = torch.tensor(embeddings_dataset[7306]["xvector"]).unsqueeze(0)
# shape (1, 512): mô tả một giọng nữ
```

Có đủ đầu vào để sinh log-mel spectrogram. Muốn ra sóng âm nghe được, cần chỉ định vocoder. Thư viện `transformers` có sẵn vocoder dựa trên **HiFi-GAN** (một mạng đối sinh GAN tạo sóng âm chất lượng cao từ spectrogram):

```
from transformers import SpeechT5HifiGan

vocoder = SpeechT5HifiGan.from_pretrained("microsoft/speecht5_hifigan")

speech = model.generate_speech(inputs["input_ids"], speaker_embeddings,
                               vocoder=vocoder)
# SpeechT5 luôn dùng tần số lấy mẫu 16 kHz
from IPython.display import Audio
Audio(speech, rate=16000)
```

Speaker embedding là gì

Là cách biểu diễn danh tính người nói gọn trong một vector cố định, bất kể câu dài ngắn. Nó dùng được cho xác minh người nói, phân tách người nói, nhận diện người nói. Hai kỹ thuật phổ biến: *I-Vectors* (dựa trên mô hình hỗn hợp Gauss) và *X-Vectors* (dùng mạng nơ-ron sâu, hiện là tiên tiến hơn). Checkpoint TTS này của SpeechT5 chỉ hỗ trợ tiếng Anh.

7.4 Bark: sinh thẳng sóng âm

Bark (Suno AI) khác SpeechT5 ở chỗ *sinh thẳng sóng âm*, không cần vocoder riêng (đã tích hợp sẵn qua bộ nén EnCodec). Bark gồm bốn mô hình: mô hình ngữ nghĩa (văn bản sang token ngữ nghĩa), mô hình âm thô, mô hình âm tinh, và bộ giải mã EnCodec. Nó rất dễ điều khiển.

```
from transformers import BarkModel, BarkProcessor

model = BarkModel.from_pretrained("suno/bark-small")
processor = BarkProcessor.from_pretrained("suno/bark-small")

# Chọn giọng từ thư viện voice preset
inputs = processor("This is a test!", voice_preset="v2/en_speaker_3")
speech_output = model.generate(**inputs).cpu().numpy()
```

Bark đa ngôn ngữ (chỉ cần đổi văn bản sang ngôn ngữ tương ứng, không cần khai báo ngôn ngữ), và tạo được giao tiếp phi ngôn ngữ (cười, thở dài) hay cả tiếng hát qua các gợi ý trong văn bản:

```
# Tiếng Pháp kèm giọng Pháp
inputs = processor("C'est un test!", voice_preset="v2/fr_speaker_1")

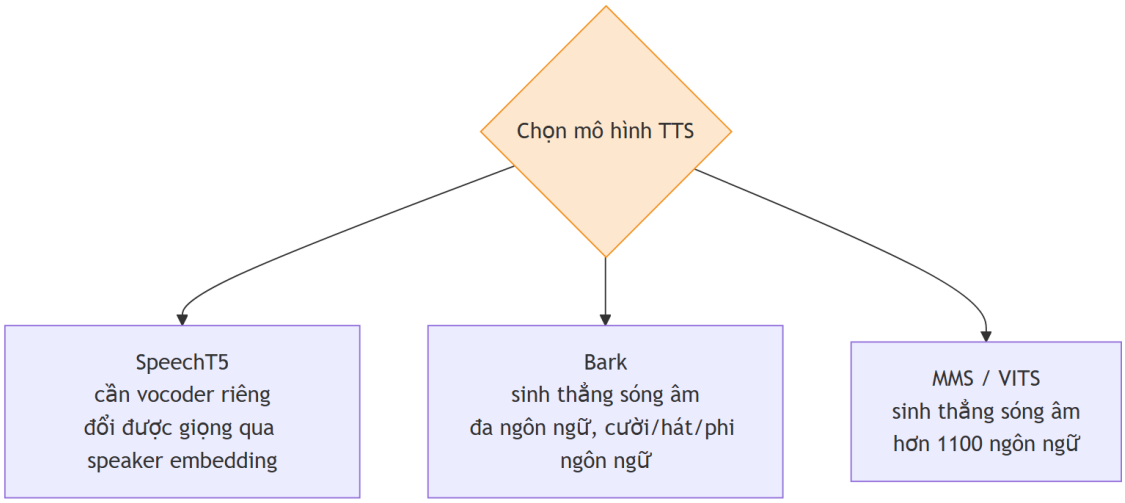
# Giao tiếp phi ngôn ngữ: [clears throat], [laughter], ...
inputs = processor(
    "[clears throat] This is a test ... and I just took a long pause.",
    voice_preset="v2/fr_speaker_1",
)

# Hát: bọc câu giữa ký hiệu nốt nhạc
inputs = processor("In the mighty jungle, I'm trying to generate barks.")
speech_output = model.generate(**inputs).cpu().numpy()
```

Bark còn hỗ trợ xử lý theo lô (nhiều câu một lúc), trên GPU có thể nhanh hơn làm tuần tự.

7.5 MMS: hơn 1 100 ngôn ngữ

Cần TTS cho ngôn ngữ không phải tiếng Anh? **Massive Multilingual Speech (MMS)** hỗ trợ tổng hợp giọng cho hơn 1 100 ngôn ngữ. MMS cho TTS dựa trên **VITS** – một mạng sinh tiếng nói đối văn bản thẳng thành sóng âm (như một bộ tự mã hoá biến phân có điều kiện), nên cũng *không cần vocoder riêng*. Ba mô hình TTS được so ở Hình 7.2.



Hình 7.2: Ba mô hình TTS tiền huấn luyện và điểm khác biệt chính.

```
from transformers import VitsModel, VitsTokenizer
import torch

# MMS cho TTS dựa trên VITS, nên dùng VitsModel/VitsTokenizer
model = VitsModel.from_pretrained("facebook/mms-tts-deu")
tokenizer = VitsTokenizer.from_pretrained("facebook/mms-tts-deu")

text_example = ("Ich bin Schnappi das kleine Krokodil, "
                "komm aus Ägypten das liegt direkt am Nil.")
inputs = tokenizer(text_example, return_tensors="pt")

with torch.no_grad():
    outputs = model(inputs["input_ids"])

speech = outputs["waveform"]
# Nghe: Audio(speech, rate=16000)
```

Muốn ngôn ngữ khác, tìm checkpoint `vits` phù hợp trên Hub.

7.6 Đánh giá mô hình TTS

Khi huấn luyện, TTS tối ưu sai số giữa spectrogram dự đoán và spectrogram đích (MSE hoặc MAE). Nhưng vì TTS là ánh xạ *một–nhiều*, giá trị mất mát đó *không* phản ánh đúng chất lượng: nhiều spectrogram khác nhau vẫn ứng với cùng một câu.

Vì vậy, đánh giá TTS dựa nhiều vào cảm nhận chủ quan của con người. Thước đo phổ biến nhất là **điểm ý kiến trung bình** (Mean Opinion Score – MOS): người nghe chấm chất lượng giọng tổng hợp theo thang 1 đến 5 qua các bài kiểm tra nghe. Khó xây thước đo khách quan vì cảm nhận giọng nói mang tính chủ quan: mỗi người nhạy khác nhau với phát âm, ngữ điệu, độ tự nhiên, độ rõ. MOS cho thông tin quý nhưng cũng kèm theo tính biến thiên và chủ quan, khiến việc so sánh các hệ TTS trở nên khó.

Tóm tắt chương

- TTS khó vì ánh xạ một–nhiều, phụ thuộc ngữ cảnh xa, và dữ liệu chất lượng đắt đỏ; dữ liệu ASR nhiều nhiễu thường không hợp.
- SpeechT5 (encoder-decoder) sinh spectrogram rồi cần vocoder HiFi-GAN; dùng speaker embedding để đổi giọng.
- Bark và MMS/VITS sinh thẳng sóng âm, không cần vocoder riêng; Bark đa năng (đa ngôn ngữ, phi ngôn ngữ, hát), MMS phủ hơn 1 100 ngôn ngữ.
- TTS khó đánh giá bằng số; thước đo chủ đạo là MOS do người nghe chấm.

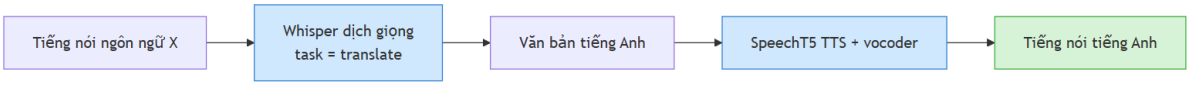
Chương 8

Ghép mọi thứ lại

Chương cuối ghép các mô hình đã học thành những ứng dụng hoàn chỉnh: dịch giọng-sang-giọng, trợ lý giọng nói, và chép lời cuộc họp. Điểm chung là *nối tầng* (cascade) nhiều mô hình sẵn có mà không cần huấn luyện thêm.

8.1 Dịch giọng-sang-giọng

Dịch giọng-sang-giọng (STST) là dịch tiếng nói từ ngôn ngữ này sang tiếng nói ngôn ngữ khác – như mở rộng của dịch máy, nhưng vào và ra đều là tiếng nói. Thay vì viết rồi dịch, bạn nói thẳng và hệ thống chuyển sang tiếng nói ngôn ngữ đích. Ta dùng cách *nối tầng* hai mô hình (Hình 8.1): Whisper dịch tiếng nói nguồn thành văn bản tiếng Anh, rồi SpeechT5 đọc văn bản đó thành tiếng nói.



Hình 8.1: Dịch giọng-sang-giọng kiểu nối tầng: dịch giọng rồi tổng hợp giọng.

Vì sao hai tầng chứ không ba

Có thể làm ba tầng (ASR chép lời cùng ngôn ngữ → dịch máy → TTS), nhưng thêm thành phần làm tăng *lan truyền lỗi* (lỗi tầng trước dồn sang tầng sau) và tăng độ trễ. Nối tầng hai mô hình đã có sẵn là cách tiết kiệm dữ liệu/tính toán để dựng STST mà không cần huấn luyện thêm.

8.1.1 Tầng 1 – dịch giọng

Dùng Whisper base (nhỏ, sinh nhanh – quan trọng vì nối hai hệ tự hồi quy). Đặt `task="translate"` để Whisper dịch sang tiếng Anh:

```
import torch
from transformers import pipeline

device = "cuda:0" if torch.cuda.is_available() else "cpu"
pipe = pipeline("automatic-speech-recognition",
                model="openai/whisper-base", device=device)

def translate(audio):
    outputs = pipe(audio, max_new_tokens=256,
                   generate_kwargs={"task": "translate"})
    return outputs["text"]
```

8.1.2 Tầng 2 – tổng hợp giọng

Ánh xạ văn bản tiếng Anh thành tiếng nói bằng SpeechT5 (chưa có `pipeline` cho TTS nên dùng mô hình trực tiếp):

```
from transformers import SpeechT5Processor, SpeechT5ForTextToSpeech, SpeechT5HifiGan
from datasets import load_dataset

processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_tts")
model = SpeechT5ForTextToSpeech.from_pretrained("microsoft/speecht5_tts").to(device)
vocoder = SpeechT5HifiGan.from_pretrained("microsoft/speecht5_hifigan").to(device)

embeddings_dataset = load_dataset("Matthijs/cmu-arctic-xvectors", split="validation")
speaker_embeddings = torch.tensor(embeddings_dataset[7306]["xvector"]).unsqueeze(0)

def synthesise(text):
    inputs = processor(text=text, return_tensors="pt")
    speech = model.generate_speech(
        inputs["input_ids"].to(device),
        speaker_embeddings.to(device), vocoder=vocoder)
    return speech.cpu()
```

8.1.3 Ghép lại và dựng demo

Nối hai hàm; đổi đầu ra sang `int16` (định dạng Gradio mong đợi):

```
import numpy as np

target_dtype = np.int16
max_range = np.iinfo(target_dtype).max

def speech_to_speech_translation(audio):
    translated_text = translate(audio)
    synthesised_speech = synthesise(translated_text)
    synthesised_speech = (synthesised_speech.numpy() * max_range).astype(np.int16)
    return 16000, synthesised_speech
```

```
import gradio as gr

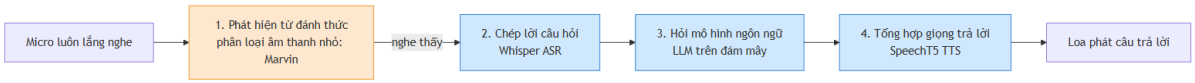
demo = gr.Blocks()
mic = gr.Interface(fn=speech_to_speech_translation,
    inputs=gr.Audio(source="microphone", type="filepath"),
    outputs=gr.Audio(label="Generated Speech", type="numpy"))
file = gr.Interface(fn=speech_to_speech_translation,
    inputs=gr.Audio(source="upload", type="filepath"),
    outputs=gr.Audio(label="Generated Speech", type="numpy"))

with demo:
    gr.TabbedInterface([mic, file], ["Microphone", "Audio File"])
demo.launch(debug=True)
```

Các nghiên cứu gần đây còn thử cách *trực tiếp* (không qua văn bản trung gian), giữ được cả ngữ điệu, cao độ của người nói gốc.

8.2 Trợ lý giọng nói Marvin

Trợ lý giọng nói (như Alexa, Siri) phản hồi khi nghe một *từ đánh thức*, rồi nghe câu hỏi và trả lời bằng giọng nói. Pipeline gồm bốn tầng, mỗi tầng một mô hình riêng (Hình 8.2).



Hình 8.2: Bốn tầng của trợ lý giọng nói.

8.2.1 Tầng 1 – phát hiện từ đánh thức

Một mô hình phân loại âm thanh nhỏ chạy liên tục trên thiết bị (nhẹ nên không hao pin). Dùng checkpoint AST tinh chỉnh trên Speech Commands – trong 35 nhãn có sẵn nhãn `"marvin"`, dùng làm từ đánh thức:

```
from transformers import pipeline
from transformers.pipelines.audio_utils import ffmpeg_microphone_live
import torch

device = "cuda:0" if torch.cuda.is_available() else "cpu"
classifier = pipeline("audio-classification",
    model="MIT/ast-finetuned-speech-commands-v2", device=device)

def launch_fn(wake_word="marvin", prob_threshold=0.5,
              chunk_length_s=2.0, stream_chunk_s=0.25):
    sampling_rate = classifier.feature_extractor.sampling_rate
    mic = ffmpeg_microphone_live(sampling_rate=sampling_rate,
        chunk_length_s=chunk_length_s, stream_chunk_s=stream_chunk_s)
    print("Listening for wake word...")
    for prediction in classifier(mic):
        prediction = prediction[0]
        if prediction["label"] == wake_word and prediction["score"] > prob_threshold:
            return True
```

Hàm `ffmpeg_microphone_live` đưa từng đoạn âm thanh nhỏ cho mô hình phân loại. Dùng ngưỡng xác suất để tránh báo nhầm khi đầu vào chỉ là nhiễu (lúc đó mọi nhãn đều điểm thấp).

8.2.2 Tầng 2 – chép lời câu hỏi

Dùng Whisper base tiếng Anh; chép "gần thời gian thực" bằng cách đưa từng đoạn nhỏ:

```
import sys

transcriber = pipeline("automatic-speech-recognition",
                       model="openai/whisper-base.en", device=device)

def transcribe(chunk_length_s=5.0, stream_chunk_s=1.0):
    sampling_rate = transcriber.feature_extractor.sampling_rate
    mic = ffmpeg_microphone_live(sampling_rate=sampling_rate,
        chunk_length_s=chunk_length_s, stream_chunk_s=stream_chunk_s)
    print("Start speaking...")
    for item in transcriber(mic, generate_kwargs={"max_new_tokens": 128}):
        sys.stdout.write("\033[K")
        print(item["text"], end="\r")
        if not item["partial"][0]:
```

```
            break
    return item["text"]
```

`stream_chunk_s` nhỏ thì "thời gian thực" hơn nhưng kém chính xác (ít ngữ cảnh). Ở đây ta dừng ghi sau `chunk_length_s` giây; thực tế có thể dùng mô hình phát hiện hoạt động giọng nói (VAD) để biết khi nào người dùng ngừng nói.

8.2.3 Tầng 3 – hỏi mô hình ngôn ngữ

Câu hỏi nhỏ nhưng LLM rất lớn, nên gọi LLM đặt trên đám mây qua Inference API cho nhanh. Ví dụ dùng một mô hình đã tinh chỉnh theo chỉ dẫn (instruct):

```
from huggingface_hub import HfFolder
import requests

def query(text, model_id="tiiuae/falcon-7b-instruct"):
    api_url = f"https://api-inference.huggingface.co/models/{model_id}"
    headers = {"Authorization": f"Bearer {HfFolder().get_token()}"}
    response = requests.post(api_url, headers=headers, json={"inputs": text})
    return response.json()[0]["generated_text"][len(text) + 1:]
```

8.2.4 Tầng 4 – tổng hợp giọng trả lời

Dùng lại hàm `synthesise` với SpeechT5 ở Mục 8.1.2. Cuối cùng ghép bốn tầng:

```
launch_fn()                       # chờ từ đánh thức
transcription = transcribe()      # chép câu hỏi
response = query(transcription)   # hỏi LLM
audio = synthesise(response)      # đọc câu trả lời

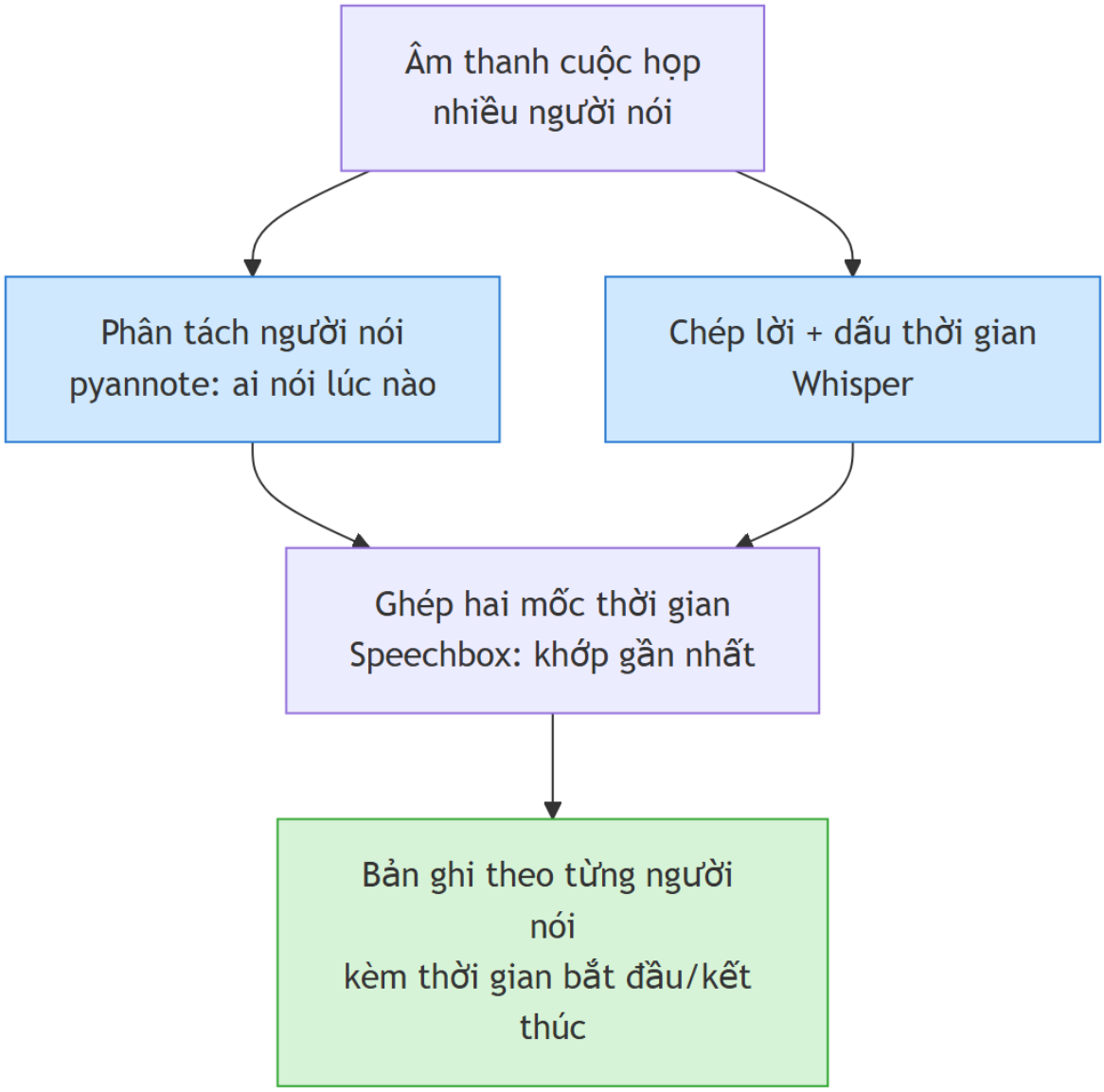
from IPython.display import Audio
Audio(audio, rate=16000, autoplay=True)
```

Cải tiến và tổng quát hoá

Có thể dùng mô hình phân loại nhị phân nhẹ hơn (chỉ xét có/không từ đánh thức), nạp mô hình đúng lúc cần để tiết kiệm điện, thêm VAD để biết khi nào ngừng nói. Thay LLM và TTS bằng *Transformers Agents* còn cho đầu ra đa phương thức (sinh ảnh, chú thích, đọc chú thích…) từ cùng một câu lệnh giọng nói.

8.3 Chép lời cuộc họp

Ghép Whisper với một mô hình **phân tách người nói** để dự đoán "ai nói lúc nào". Khớp dấu thời gian của hai mô hình, ta có bản ghi cuộc họp phân theo từng người nói kèm thời gian – phiên bản cơ bản của các dịch vụ như Otter.ai (Hình 8.3).



Hình 8.3: Chép lời cuộc họp: phân tách người nói + chép lời rồi ghép thời gian.

8.3.1 Phân tách người nói

`transformers` chưa có mô hình phân tách người nói, nhưng dùng được checkpoint `pyannote.audio` từ Hub. Nó dự đoán mốc bắt đầu/kết thúc cho mỗi lượt nói:

```
pip install --upgrade pyannote.audio
```

```
from pyannote.audio import Pipeline
import torch

diarization_pipeline = Pipeline.from_pretrained(
    "pyannote/speaker-diarization@2.1", use_auth_token=True)

# pyannote cần tensor PyTorch dạng (kênh, độ dài)
input_tensor = torch.from_numpy(sample["audio"]["array"][None, :]).float()
outputs = diarization_pipeline(
    {"waveform": input_tensor, "sample_rate": sample["audio"]["sampling_rate"]})
```

```
# [{segment: {start: 0.5, end: 14.5}, label: SPEAKER_01},
#  {segment: {start: 15.4, end: 21.4}, label: SPEAKER_00}]
```

8.3.2 Chép lời kèm dấu thời gian

Whisper với `return_timestamps=True` cho mốc thời gian từng đoạn:

```
from transformers import pipeline

asr_pipeline = pipeline("automatic-speech-recognition", model="openai/whisper-base")
asr_pipeline(sample["audio"].copy(),
             generate_kwargs={"max_new_tokens": 256},
             return_timestamps=True)
# Mỗi "chunk": {timestamp: (start, end), text: ...}
```

8.3.3 Khớp hai mốc thời gian với Speechbox

Mốc của hai mô hình không khớp hoàn hảo. Thư viện **Speechbox** tìm ranh giới gần nhau nhất (cực tiểu khoảng cách tuyệt đối) và cắt bản ghi theo người nói:

```
pip install git+https://github.com/huggingface/speechbox
```

```
from speechbox import ASRDiarizationPipeline

pipeline = ASRDiarizationPipeline(
    asr_pipeline=asr_pipeline, diarization_pipeline=diarization_pipeline)

outputs = pipeline(sample["audio"].copy())
# [{speaker: SPEAKER_01, text: "...", timestamp: (0.0, 15.48)},
#  {speaker: SPEAKER_00, text: "...", timestamp: (15.48, 21.28)}]
```

Có thể định dạng lại cho đẹp (mỗi người một đoạn, kèm thời gian làm tròn) bằng vài hàm trợ giúp nhỏ. Thuật toán khớp theo khoảng cách nhỏ nhất tuy đơn giản nhưng chạy tốt trong thực tế.

Tóm tắt chương

- Nối tầng các mô hình sẵn có là cách nhanh, tiết kiệm để dựng ứng dụng mới mà không cần huấn luyện thêm.
- Dịch giọng-sang-giọng = Whisper (dịch) + SpeechT5 (tổng hợp); cân nhắc lan truyền lỗi và độ trễ khi thêm tầng.
- Trợ lý giọng nói gồm bốn tầng: phát hiện từ đánh thức, chép lời, hỏi LLM, tổng hợp giọng.
- Chép lời cuộc họp ghép Whisper (chép + dấu thời gian) với phân tách người nói (pyannote), khớp thời gian bằng Speechbox.

Chương 9

Case study: kiểm tra đọc đúng script

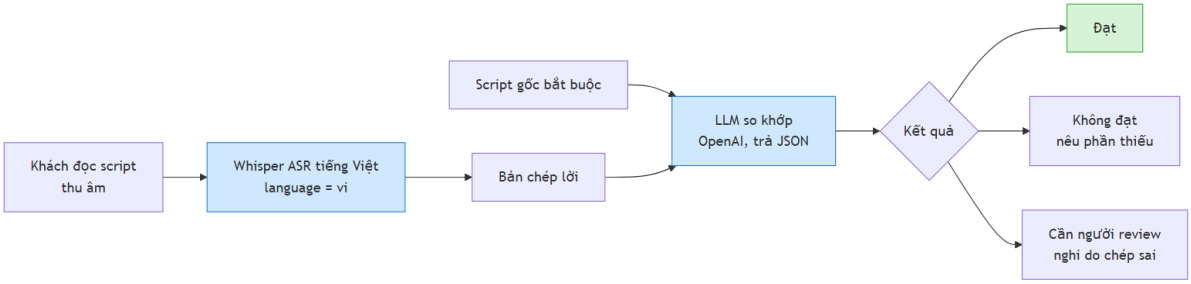
Chương này là một **tình huống thực chiến** ngoài khoá học gốc: trong quy trình bán bảo hiểm, khách hàng phải đọc một *script* bắt buộc (xác nhận đã hiểu quyền lợi, điều khoản, đồng ý tham gia). Hệ thống thu âm, chép lời bằng nhận dạng tiếng nói, rồi kiểm tra xem khách có đọc *đúng và đủ* nội dung không. Ta ghép Whisper (đã học ở Chương 6) với một mô hình ngôn ngữ lớn (LLM) làm khâu so khớp.

Phạm vi và nguồn

Case study này *không* nằm trong khoá *Audio Course* của Hugging Face – đây là ví dụ ứng dụng mở rộng, dựng lại từ kiến thức các chương trước. Code dùng thư viện thật và giữ cú pháp chạy được. Các **ngưỡng** (độ dài, mức chấp nhận) để dạng tham số kèm giải thích; bạn phải tự hiệu chỉnh trên dữ liệu thật, tài liệu không đưa ra con số như thể đã đo.

9.1 Phát biểu bài toán

Đầu vào là một đoạn ghi âm khách đọc script; script gốc đã biết trước và cố định. Đầu ra là một phán quyết: *đạt* (đọc đúng và đủ) hay *không đạt* (thiếu/sai nội dung bắt buộc), kèm lý do. Luồng xử lý ở Hình 9.1.



Hình 9.1: Luồng kiểm tra đọc đúng script: chép lời rồi so khớp bằng LLM.

Điểm mấu chốt cần nhận ra: bài toán gồm *hai khâu tách biệt*, và chúng khó ở hai chỗ khác nhau.

- **Khâu nhận dạng** (Whisper): biến tiếng nói thành chữ. Khó ở độ chính xác chép lời tiếng Việt, nhất là qua điện thoại nhiều nhiễu.
- **Khâu so khớp** (LLM): quyết định bản chép có khớp script không. Khó ở chỗ khách *không bao giờ* đọc y hệt – có "à, ờ", đổi "tôi" thành "tui", đảo trật tự câu – nhưng vẫn phải tính là đạt nếu đủ ý.

Không cần huấn luyện mô hình

Cả hai khâu đều dùng **mô hình tiền huấn luyện gọi qua API**. Whisper lo chép lời, LLM lo so khớp. Khâu so khớp *không* phải là bài toán học máy cần train – nó chỉ là so sánh bản chép với một script biết trước. Vì vậy, trong phần lớn trường hợp, bạn **không cần thu audio của khách để train** gì cả.

9.2 Khâu 1 – Nhận dạng tiếng nói tiếng Việt

Tiếng Việt (`vi`) nằm trong gần 96 ngôn ngữ Whisper được tiền huấn luyện, nên chạy được ngay mà không cần làm gì thêm. Chỉ cần khai báo ngôn ngữ khi gọi:

```
import torch
from transformers import pipeline

device = "cuda:0" if torch.cuda.is_available() else "cpu"
asr = pipeline("automatic-speech-recognition",
               model="openai/whisper-large-v3", device=device)

def transcribe(audio_path):
    out = asr(audio_path,
              generate_kwargs={"task": "transcribe", "language": "vi"})
    return out["text"]
```

9.2.1 Chọn cỡ mô hình và chuẩn bị âm thanh

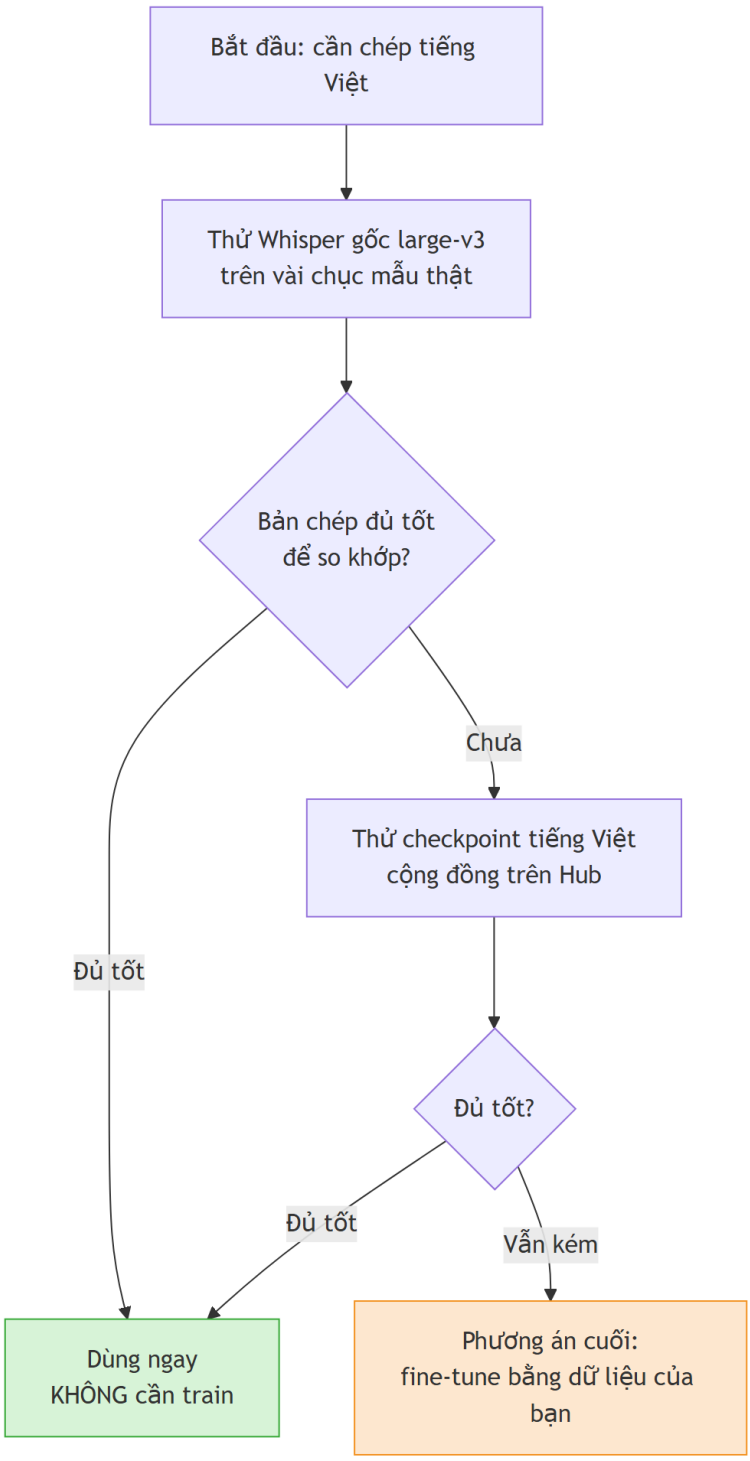
Tiếng Việt có dấu thanh, mô hình nhỏ (`tiny`, `base`) hay sai dấu và sai từ. Với bài toán pháp lý cần độ chính xác cao, nên ưu tiên `large-v3` (hoặc `medium` nếu cần nhẹ hơn). Lưu ý về âm thanh:

- **Tần số lấy mẫu**: Whisper làm việc ở 16 kHz. Âm thanh tổng đài thường 8 kHz nên cần tăng mẫu (xem Chương 2); micro trực tiếp thường đã 16 kHz trở lên.
- **Số kênh**: trộn về một kênh (mono) trước khi đưa vào mô hình.
- **Âm thanh dài**: nếu script dài quá 30 giây, bật cắt đoạn (`chunk_length_s=30`) như ở Chương 6.

Trên Hub còn có các checkpoint tiếng Việt do cộng đồng tinh chỉnh (lọc theo tác vụ nhận dạng tiếng nói và ngôn ngữ `vi`); nhiều bản chép tốt hơn Whisper gốc cho giọng vùng miền. Có thể thay thẳng vào tham số `model`.

9.2.2 Khi nào cần fine-tune, khi nào không

Đừng vội fine-tune. Hãy đi theo thứ tự ở Hình 9.2: thử Whisper gốc trước, rồi checkpoint cộng đồng, chỉ khi cả hai vẫn kém mới tính tinh chỉnh.



Hình 9.2: Trình tự quyết định: đa số trường hợp không cần huấn luyện lại.

Chỉ nên fine-tune khi Whisper gốc sai *một cách hệ thống* trong điều kiện của bạn: nhiều thuật ngữ bảo hiểm riêng, tên sản phẩm lạ, giọng vùng miền nặng, hoặc nhiễu đường truyền. Lúc đó áp đúng quy trình tinh chỉnh Whisper ở Chương 6 với dữ liệu đã thu. Đây là *phương án cuối*.

Lỗi có thể không nằm ở ASR

Đôi khi khách đọc đúng nhưng Whisper chép sai một từ, khiến khâu sau tưởng là thiếu nội dung. Nhiều lúc vấn đề thật ra ở khâu *so khớp* chứ không phải ASR. Vì vậy hãy đánh giá chất lượng chép lời (ví dụ bằng WER trên vài chục mẫu có bản chép tay chuẩn, như ở Chương 6) *trước khi* kết luận cần train lại.

9.3 Khâu 2 – So khớp script bằng LLM

Vì sao không so khớp bằng cách tính tỉ lệ lỗi từ (WER) giữa bản chép và script? Vì WER phạt nặng mọi sai khác bề mặt: khách thêm "à ờ" hay đổi cách xưng hô là WER tăng dù nội dung vẫn đủ. Một **LLM** (ví dụ của OpenAI) đánh giá được theo *ý nghĩa*: "khách có nói đủ các ý bắt buộc không", đồng thời vẫn kiểm được những cụm phải đọc chính xác tuyệt đối.

9.3.1 Hai kiểu kiểm tra

Trong một script thường có hai loại nội dung, cần đối xử khác nhau (Bảng 9.1).

Kiểu	Ví dụ	Cách kiểm
Phải đọc chính xác	Câu đồng ý pháp lý ("tôi đồng ý tham gia"), tên sản phẩm, số hợp đồng	Đối chiếu gần như nguyên văn; sai là không đạt
Chỉ cần đủ ý	Giải thích quyền lợi, nhắc thời gian cân nhắc	Cho phép diễn đạt khác, miễn truyền đạt đủ nội dung

Bảng 9.1: Hai kiểu nội dung trong script và cách kiểm tra tương ứng.

9.3.2 Gọi LLM và yêu cầu trả JSON có cấu trúc

Nguyên tắc quan trọng nhất: bắt LLM trả về **JSON có cấu trúc** thay vì văn xuôi, để hệ thống đọc máy được và dễ kiểm toán. Đặt nhiệt độ bằng 0 cho ổn định, và ràng buộc LLM chỉ đánh giá dựa trên bản chép + script, không suy diễn thêm.

```
import json
from openai import OpenAI

client = OpenAI()   # đọc khoá từ biến môi trường OPENAI_API_KEY

SYSTEM_PROMPT = """Bạn là trợ lý kiểm tra tuân thủ cho quy trình bán bảo hiểm.
Nhiệm vụ: so bản chép lời (do nhận dạng tiếng nói tạo ra) với SCRIPT bắt buộc.
Chỉ đánh giá dựa trên hai văn bản được cung cấp, KHÔNG suy diễn thêm.

Quy tắc:
- Các cụm "phải đọc chính xác" phải xuất hiện gần như nguyên văn.
- Các nội dung "chỉ cần đủ ý" được phép diễn đạt khác, miễn đủ nội dung.
- Bỏ qua từ đệm (à, ờ, ừm), khác biệt xưng hô (tôi/tui) và lỗi chính tả nhỏ.
- Nếu nghi bản chép sai do nhận dạng (chứ không phải người đọc thiếu),
  hãy đánh dấu can_review = true để con người xem lại.

Trả về DUY NHẤT một JSON đúng cấu trúc sau, không kèm giải thích ngoài JSON:
{
  "ket_qua": "dat" | "khong_dat",
  "cac_y_bat_buoc_thieu": [chuỗi, ...],
  "cac_cum_chinh_xac_sai": [chuỗi, ...],
```

```
  "can_review": true | false,
  "ly_do": "giải thích ngắn gọn"
}"""

def check_script(transcript, script, cum_chinh_xac):
    user_msg = (
        f"SCRIPT:\n{script}\n\n"
        f"CÁC CỤM PHẢI ĐỌC CHÍNH XÁC:\n- " + "\n- ".join(cum_chinh_xac) + "\n\n"
        f"BẢN CHÉP LỜI:\n{transcript}"
    )
    resp = client.chat.completions.create(
        model="gpt-4o",
        temperature=0,
        response_format={"type": "json_object"},   # ép trả JSON
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_msg},
        ],
    )
    return json.loads(resp.choices[0].message.content)
```

9.3.3 Ghép hai khâu

Nối Whisper và LLM thành một hàm hoàn chỉnh:

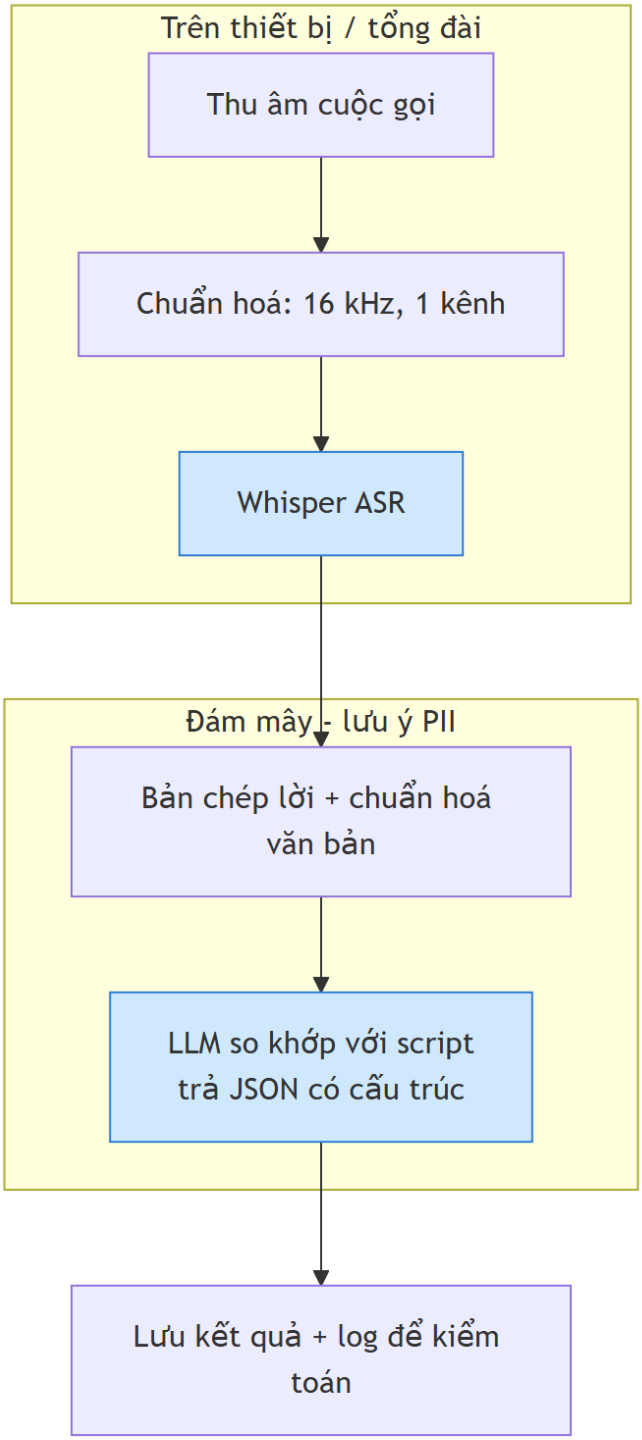
```
SCRIPT = (
    "Tôi xác nhận đã được tư vấn đầy đủ về quyền lợi và điều khoản. "
    "Tôi hiểu thời gian cân nhắc 21 ngày. Tôi đồng ý tham gia hợp đồng."
)
CUM_CHINH_XAC = ["tôi đồng ý tham gia", "thời gian cân nhắc 21 ngày"]

def verify(audio_path):
    transcript = transcribe(audio_path)
    ket_qua = check_script(transcript, SCRIPT, CUM_CHINH_XAC)
    return {"transcript": transcript, **ket_qua}

result = verify("ghi_am_khach.wav")
# {
#   "transcript": "ờ tôi xác nhận đã được tư vấn đầy đủ ...",
#   "ket_qua": "dat",
#   "cac_y_bat_buoc_thieu": [],
#   "cac_cum_chinh_xac_sai": [],
#   "can_review": false,
#   "ly_do": "Đã đọc đủ các ý và các cụm bắt buộc."
# }
```

9.4 Kiến trúc triển khai và lưu ý vận hành

Khi đưa vào thực tế, kiến trúc thường như Hình 9.3: ASR có thể chạy ngay trên thiết bị/tổng đài để giảm độ trễ và hạn chế gửi âm thanh ra ngoài; chỉ *bản chép* (nhẹ hơn nhiều) mới gửi lên LLM đám mây.



Hình 9.3: Kiến trúc triển khai: ASR tại chỗ, so khớp bằng LLM trên đám mây.

Quyền riêng tư và dữ liệu cá nhân (PII)

Ghi âm bảo hiểm chứa dữ liệu cá nhân nhạy cảm (họ tên, số hợp đồng, sức khoẻ). Gửi bản chép sang API bên thứ ba là **chuyển dữ liệu khách hàng ra ngoài** – phải kiểm tra quy định tuân thủ và hợp đồng xử lý dữ liệu trước. Cân nhắc: che/ẩn danh thông tin nhạy cảm trong bản chép trước khi gửi; hoặc dùng LLM chạy nội bộ nếu quy định yêu cầu.

Vài lưu ý vận hành khác:

- **Để con người quyết định ca khó**: khi `can_review` là đúng (nghi lỗi do chép sai) hoặc điểm sát ngưỡng, chuyển cho người kiểm tra thay vì tự động đánh trượt. Đây là mô thức "con người trong vòng lặp" (human-in-the-loop).
- **Chọn ngưỡng và tiêu chí** phải dựa trên dữ liệu thật của bạn: thu một tập mẫu có dán nhãn đạt/không đạt do người đánh giá, rồi dò ngưỡng. Tài liệu này không áp con số sẵn.
- **Lưu vết để kiểm toán**: lưu lại bản chép, JSON kết quả và phiên bản script/mô hình cho mỗi lần kiểm, phục vụ đối soát về sau.
- **Chi phí và độ trễ**: ASR là phần nặng nhất; nếu xử lý số lượng lớn, cân nhắc GPU hoặc dịch vụ inference. Phần gọi LLM nhẹ vì chỉ gửi văn bản.

Tóm tắt chương

- Bài toán gồm hai khâu: Whisper chép lời tiếng Việt, LLM so khớp với script; cả hai đều là mô hình tiền huấn luyện gọi qua API, **không cần train**.
- Whisper hỗ trợ tiếng Việt sẵn; dùng `large-v3` cho độ chính xác, hoặc checkpoint cộng đồng; chỉ fine-tune khi sai hệ thống.
- Dùng LLM thay cho WER vì khách không đọc y hệt script; tách rõ "phải đọc chính xác" và "chỉ cần đủ ý", và ép LLM trả JSON có cấu trúc.
- Vận hành: ASR tại chỗ, so khớp trên đám mây; chú ý PII, giữ con người trong vòng lặp cho ca khó, và lưu vết để kiểm toán.